

ELF: 嵌入式语言流

(英文原标题: ELF: Embedded Language Flows)

Keya Hu* Linlu Qiu* Yiyang Lu Hanhong Zhao Tianhong Li Yoon Kim Jacob Andreas Kaiming He
麻省理工学院 (MIT)

*共同一作; 作者排序由抛硬币决定。代码: <https://github.com/lillian039/ELF>

arXiv:2605.10938v2 [cs.CL] 2026 年 6 月 26 日

摘要

扩散模型与基于流的模型已成为生成连续数据（如图像、视频等领域的标准方法）的既定范式。它们的成功吸引了越来越多的人将其应用于语言建模。与图像领域的同类模型不同，当前领先的扩散语言模型（DLM）主要在离散词元（discrete token）上运行。在本文中，我们证明只要对离散域做极少的适配，连续 DLM 也可以被做得非常有效。我们提出了「嵌入式语言流」（Embedded Language Flows, ELF）——一类基于连续时间流匹配（Flow Matching）的、运行在连续嵌入空间中的扩散模型。与已有的 DLM 不同，ELF 在绝大部分时间里都停留在连续嵌入空间中，直到最后一个时间步，才用一个共享权重的网络将其映射回离散词元。这种形式化让作者可以直接迁移图像域扩散模型中那些成熟的技术，例如无分类器引导（CFG）。实验表明，ELF 明显优于领先的离散与连续 DLM，能以更少的采样步数取得更好的生成质量。这些结果表明，ELF 为构建有效的连续 DLM 提供了一条很有前景的路径。



图 1: ELF 在更少的采样步数下取得了更低的生成困惑度（Generative Perplexity, Gen. PPL），且无需蒸馏。ELF 在做到这一点的同时，仅使用了 10 倍少的训练词元。（模型规模：ELF 为 105M，其余为 170M；数据集：OWT。详细对比见图 7。）

1 引言

扩散模型 [72, 73, 30] 与基于流的模型 [42, 43, 3] 已成为生成连续数据的核心范式，在合成图像、视频以及其他连续域数据方面表现出强大的能力。这些进展促使人们越来越多地探索把扩散方法引入语言建模，催生了关于扩散语言模型（DLM）的大量研究。DLM 通常有两种形式化方式：连续式或离散式。连续式 DLM 将离散词元映射为连续表示，并在由此得到的连续空间中做去噪 [39, 15, 21]。相反，离散式 DLM 直接在词元空间中运行，并对离散随机变量构造概率扩散模型 [5, 27, 45, 63, 64]。近年来 DLM 的进展大多集中在离散范式，很大程度上是因为离散 DLM 具有更强的实证性能 [38, 54, 85, 65]。但连续 DLM 当前存在的性能差距，究竟源于语言建模固有的离散本质，还是源于尚未被充分探索的算法设计选择，仍然是一个悬而未决的问题。

在本工作中，我们提出「嵌入式语言流」（ELF）——一类基于流匹配 [42, 43, 3] 的连续 DLM。ELF 在两层意义上是「连续」的。首先，它运行在

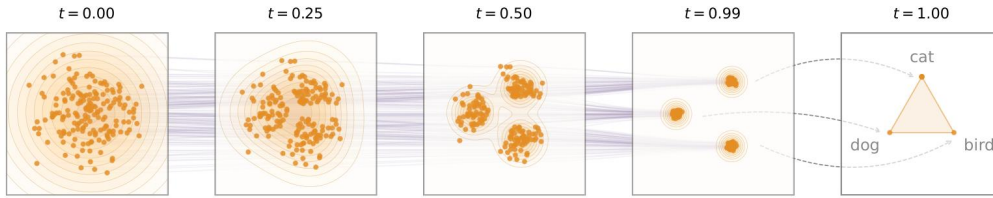


图 2：ELF 的概念示意。橙色点表示表示在连续嵌入空间中的数据，紫色线条表示从高斯噪声到干净嵌入的去噪轨迹。离散化只在最后一个时间步（ $t = 1$ ）使用共享权重网络进行。

连续嵌入空间中，通过在整个流动过程中直接对连续表示做去噪，只在最后一个时间步才考虑离散化。其次，它采用连续时间的形式化，遵循流匹配 [42, 43, 3]，这使我们能够通过时间导数来定义速度场。这一形式使得 ELF 能够受益于流匹配的最新进展，而流匹配如今被广泛用于实例化图像与视频生成中的扩散模型 [48, 16, 7, 79]。

沿用潜在扩散模型（LDM）[61] 的做法，ELF 通过对输入离散词元施加一个编码器模型来构造连续嵌入空间。该编码器可以预先训练、联合训练，或者冻结为随机权重。与潜在扩散不同，ELF 不需要单独的解码器，因此在推理时不会引入额外的组件。这一设计基于如下观察：流匹配中最后一个时间步可以自然地复用来把连续嵌入映射回离散词元，从而免去显式解码器。这样，一个共享权重的网络被训练来在除最后一步之外的所有步做去噪，并在最后一步做解码（即离散化）（见图 2）。

ELF 借鉴了此前的连续 DLM，但追求一种极简设计，以解决连续与离散空间之间的接口问题。与连续 DLM 的开创性工作[39, 15, 21] 以及许多其他采用逐步离散化损失（如交叉熵）的方法不同，ELF 在几乎所有的步骤中都在连续嵌入空间里做去噪，从而为流动力学提供了最大限度的灵活性。而与通常运行在压缩潜在空间、依赖单独解码器的潜在扩散方法 [46, 50, 70] 不同，ELF 直接运行在高维潜在空间 [37] 中，并且不需要额外解码器。

实验上，我们表明 ELF 在这些工作所建立的评测协议下，优于离散 DLM 的领先方法以及已有的连续 DLM（图 1）。ELF 以比领先的离散 DLM（如 MDLM [63] 和 Duo [64]）以及同期出现的连续 DLM（如 FLM [35] 和 LangFlow [11]）更少的采样步数，取得了更好的生成质量。此外，ELF 仅用 10 倍少的训练词元、且无需任何蒸馏就实现了上述性能。我们进一步展示 ELF 在机器翻译 [8] 与摘要 [52] 上表现强劲。总体而言，这些结果表明连续 DLM 可以极有竞争力，同时只需要对离散化做极少的处理，为基于扩散的语言建模提供了一条有前景的方向。

2 背景与相关工作

扩散/流式模型。

扩散模型 [72, 30, 73] 与流式模型 [42, 43, 2] 通过常微分方程或随机微分方程（ODE/SDE）把噪声变换为数据。在 DDPM 风格的形式化中，生成由相邻状态之间的转移定义 [72, 30, 53]，这些状态可以是离散的或连续的。离散状态需要类别转移分布，如离散 DLM [5, 63] 那样；连续状态则通常在高斯扰动下通过分数或噪声预测来建模 [73, 30, 16]。流匹配通过学习沿某条连续路径的速度场，把这一视角推广到连续时间 [42, 43, 2]，其中噪声、数据与速度预测可以互相重参数化 [16, 37]。我们的方法采用流匹配，在连续嵌入空间与连续时间中形式化语言生成。

连续扩散语言模型。

连续 DLM 把离散词元映射到连续空间以做去噪。嵌入空间方法，如 Diffusion-LM [39]、CDCD [15] 和 DiffuSeq [21]，直接把高斯噪声加到词元嵌入上 [75, 88, 24, 81, 86, 41, 83, 17]，后续的 FlowSeq [31] 则在同一个嵌入空间里用流匹配替代扩散过程。另一个互补的方向研究基于单纯形（simplex）的表示，包括

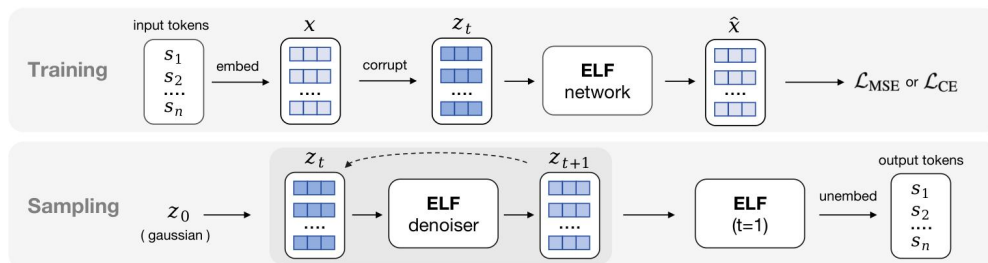


图 3：训练阶段，离散词元被编码为干净嵌入 x 并扰动为 z_t ，ELF 用它来预测 $\hat{x}$ 。模型用去噪损失 $\mathcal{L}_{MSE}$ 或逐词元交叉熵损失 $\mathcal{L}_{CE}$ 训练。推理阶段，ELF 从高斯噪声 z_0 出发，迭代地把嵌入从 z_t 去噪到 z_{t+1} 。只有到最后一步，ELF 才切换为解码模式，通过一个反嵌入（unembedding）层把最终嵌入投影回离散词元。

SSD-LM [26] 与 TESS [49, 77]，以及相关的流形方法 [32, 12] 和模拟位编码（对词元的连续二进制码做去噪）[6]。尽管这些方法为离散词元提供了连续松弛，但它们的轨迹往往通过单纯形约束、词元对齐的位编码以及词元级交叉熵目标等机制，仍旧与离散词元空间绑定在一起。相比之下，ELF 完全在连续嵌入空间中做去噪，不施加逐步词元级监督，只在最后一步做离散化。

另一条脉络把潜在扩散应用到冻结的编码器表示上，由 LD4LG [46] 及其后续工作 [90, 67, 47, 50, 70] 代表，最近的工作进一步把潜在空间与扩散模型联合学习 [51, 25]。与上述许多扩散方法类似，这些方法通常遵循 DDPM 风格或基于分数的形式化，配合 DDPM 噪声调度 [30, 53]，并且额外依赖一个单独的解码器网络来恢复词元，该解码器要么在独立阶段训练，要么与扩散模型联合训练 [51]。相比之下，ELF 采用连续时间流匹配形式化，使用线性（rectified-flow）插值路径 [42, 43, 2]，并且不需要单独解码器。这把基于流的训练与采样引入语言扩散，使 ELF 能够受益于流匹配的最新进展。

若干同期工作也重新审视了连续的、基于流的语言建模。DFM [57]、CFM [62]、FLM/FMLM [35] 和 LangFlow [11] 都在流动轨迹上加入了词元级交叉熵监督，不过它们在连续状态空间上有所不同，包括单纯形空间、one-hot 词元编码以及嵌入空间。其中一些方法还引入了蒸馏以实现少步生成，例如蒸馏后的 DFM/CFM 和 FMLM。相比之下，ELF 把去噪轨迹完全保持在无约束的连续嵌入空间中，只在最后的解码步骤施加词元级监督。更全面的综述见附录 A。

离散扩散语言模型。

由于语言的离散本质，另一条脉络直接在词元空间中施加扩散。D3PM [5] 定义了通用的离散扰动过程，包括吸收式（absorbing）与均匀式（uniform）转移。掩码扩散模型，如 MDLM [63]，使用一个特殊的 [MASK] 吸收态，并通过迭代式去掩码来生成样本 [27, 54, 85, 71]。后续工作通过再掩码、自适应推断 [80, 82] 以及半自回归块扩散（如 E2D2 [4]）来改进采样与效率。均匀态扩散模型，如 Duo [64]，则把词元扩散到均匀类别分布，从而在推理时能反复修订词元 [64, 14, 65]。近来的研究进一步扩展了离散 DLM 的规模，并将其扩展到代码与多模态生成 [23, 74, 84, 87, 36]。另一条相关工作通过在离散

与连续空间中联合去噪，来衔接离散与连续扩散 [22, 91, 58, 92]。总体而言，离散扩散模型目前仍是基于扩散的语言建模中的主导范式 [38]。

3 嵌入式语言流

本节给出我们用于语言建模的、基于流的形式化（图 3）。我们的方法利用流模型的迭代特性，主要在连续嵌入空间中做去噪，只在最后一步把干净嵌入转回离散词元。沿用先前的工作 [63, 64, 35, 11]，我们在更简单的无条件生成设定下描述本方法。该框架可以扩展到条件生成，详见 3.3 节。

3.1 ELF 框架

从离散词元到连续嵌入。

要把连续扩散应用于语言，我们首先把离散词元映射到连续表示。给定一个句子，把它分词为词元序列 $s = [s_1, \dots, s_L] \in V^L$ ，其中每个 s_i 取自词表 V ， L 为序列长度。然后我们把离散词元序列映射到一个连续嵌入空间。嵌入方法的选择是灵活的。默认情况下，我们使用预训练的 T5 编码器 [60] 来获得双向上下文嵌入。我们还探索了其他联合训练与随机化的嵌入（见 4.1 节）。编码器只在训练时使用，不会在推理时带来额外模块。

对连续嵌入做流匹配。

得到连续语言表示后，我们使用流匹配 [42, 43, 3] 在所得的嵌入空间中形式化去噪过程。流匹配在该空间中定义了从噪声到数据的连续流动路径。令 $x \sim p_{\text{data}}(x)$ 表示嵌入分布， $\epsilon \sim p_{\text{noise}}(\epsilon)$ 表示噪声分布（例如 $\epsilon \sim N(0, I)$ ）。带噪潜伏变量由线性插值（“rectified flows”）定义： $z_t = t x + (1 - t) \epsilon$ ，其中 $t \in [0, 1]$ ， $z_0 \sim p_{\text{noise}}$ ， $z_1 \sim p_{\text{data}}$ 。在连续时间中，流动速度 v 定义为 z 关于时间的导数，即 $v = dz/dt = x - \epsilon$ 。

标准的流匹配直接用神经网络参数化 v ，而 ELF 则遵循图像生成领域的最新进展，改为参数化 x [37]（ x -prediction）。具体而言，令 $x_\theta = \text{net}_\theta(z_t, t)$ 表示网络的即时输出。我们通过最小化预测速度与目标速度之间的均方误差（MSE）来训练模型：

$v_\theta(z_t, t)$ denote the network’s immediate output. We train the model by minimizing the mean squared error (MSE) between the predicted velocity and the target velocity:

$$\mathcal{L}_{\text{MSE}} = \mathbb{E}_{t, x, \epsilon} \|v_\theta(z_t, t) - v\|^2 = \mathbb{E}_{t, x, \epsilon} \frac{1}{(1 - t)^2} \|x_\theta(z_t, t) - x\|^2,$$

leverage the relation $v(z_t, t) = (x - z_t)/(1 - t)$ [37].

（式 1）

其中我们利用了关系 $v(z_t, t) = (x - z_t)/(1 - t)$ [37]。

x -prediction 的参数化对 ELF 很重要。首先，它使流匹配能在高维表示（例如每词元 768 维嵌入）上有效发挥作用，这与 [37] 中的观察一致（ELF 关于预测目标的消融见附录 D.1）。其次，预测干净嵌入（即 x ）与在最后一步预测干净离散词元的目标天然对齐（下文将讨论），而标准的 v -prediction 则不能。尽管 v 可以由网络预测并变换为 x ，但把去噪（MSE 损失）与解码（交叉熵损失）两个目标绑定起来的权重共享会因此受损。实证上我们观察到，当权重与最后的离散化步骤共享时， v -prediction 表现很差。

回到离散词元。

由于生成输出由离散词元构成，我们在最后一个时间步（即 $t = 1$ ）把干净嵌入转回词元。通过把 ELF 的最后一个时间步自然地视为连续到离散的解码，我们的方法不需要单独的解码器（等价地，可以看作一个与去噪器共享权重的解码器）。

该时间步的网络输入应当是极限 $t \rightarrow 1$ 时的 z_t 。但由于当 $t \rightarrow 1$ 时 $z_t \rightarrow x$ ，我们在这个最后一步引入一个词元级的扰动过程，以产生一个非平凡的训练输入，记为 $\tilde{z}$ （详见附录 C.1）。同一个网络 net_θ 把 $\tilde{z}$ 映射为干净嵌入 $x_\theta(\tilde{z})$ ，随后由一个可学习的「反嵌入」矩阵 W 投影为 logits。我们针对真值词元 s 最小化逐词元交叉熵（CE）损失：

It is subsequently projected by a learnable unembedding matrix W to obtain $x_\theta(\tilde{z})$, which is then used to calculate a per-token cross-entropy (CE) loss against the ground-truth token s :

$$\mathcal{L}_{\text{CE}} = \mathbb{E}_{\tilde{z}} [\text{CrossEnt}(Wx_\theta(\tilde{z}), s)],$$

where x_θ shares weights with that in Eq. (1) and is conditioned on a binary “mode” variable.

(式 2)

网络 x_θ 与式 (1) 中的网络共享权重，并且在时间条件 $t = 1$ 之外，还以一个二值的「模式」词元（denoise 或 decode）为条件。在推理时，我们只在最后一步 $t = 1$ 计算 $Wx_\theta(z_t)$ ，并应用 argmax 得到离散词元。

3.2 伪代码

ELF 的核心概念总结在算法 1 与算法 2 中（详见附录图 10）。

Algorithm 1 ELF. training.

Two-branch computation is batched, adding no extra training cost.

```
# net(z, t, mode): ELF network
# s: a sequence of discrete tokens

x = encode(s)
if uniform(0, 1) < threshold:
    # denoising branch
    t = sample_t()
    e = randn_like(x)
    z = t * x + (1 - t) * e
    v = x - e
    x_pred = net(z, t, mode="denoise")
    v_pred = (x_pred - z) / (1 - t)
    loss = mse_loss(v_pred, v)
else:
    # decoding branch (t = 1)
    z = corrupt(x)
    x_pred = net(z, t=1, mode="decode")
    s_pred = unembed(x_pred)
    loss = ce_loss(s_pred, s)
```

Algorithm 2 ELF. inference.

We show ODE for simplicity. SDE sampler is also applicable.

```
# shape: shape of embedded sequences
# ts: sampling time schedule, from 0 to 1

z = randn(shape)
for i in range(len(ts) - 1):
    t = ts[i]
    dt = ts[i + 1] - ts[i]
    x_pred = net(z, t, mode="denoise")

    # convert x prediction to velocity
    v = (x_pred - z) / (1 - t)
    z = z + dt * v

# final step
h = net(z, t=1, mode="decode")

# unembedding
token_logits = unembed(h)
tokens = argmax(token_logits)
```

single batch, and masking is used to selectively apply the appropriate corruption and unembedding

算法 1（左）：ELF 的训练；算法 2（右）：ELF 的推理。

训练。

与标准流匹配一样，ELF 使用单一网络 net_θ 来建模所有时间步，并以 t 作为条件。这包括最后一个时间步 $t = 1$ ，它使用不同的预处理（扰动）与后处理（损失计算）。为清晰起见，我们在算法 1 中用显式的「if」分支来体现这一区别。实际中，两个分支的样本在单个 batch 中一起处理，并使用掩码来选

择性施加相应的扰动与反嵌入操作以及相应的损失项。网络还以一个二值「模式」词元为条件，指示操作是「denoise」还是「decode」。

推理。

推理时，ELF 迭代地把带噪样本变换为干净嵌入。从 $z_0 \sim N(0, I)$ 出发，ELF 求解 ODE: $dz_t/dt = v_\theta(z_t, t)$ ，用数值（如欧拉）求解器近似。在最后一个时间步 $t = 1$ ，我们在「decode」模式下运行网络，并执行反嵌入与离散化。

除了 ODE 形式化，我们的方法还支持一种受 SDE 启发的采样器。与流匹配相关联的 SDE 可参照 [48] 推导，其中动力学可以被解释为在每一步注入无穷小噪声。实际中，我们采用一个更简单的近似来模拟这一行为：在每一步注入小噪声，同时把时间变量 t 移向噪声更强的区间（详见附录，算法 6）。为简洁起见，我们把得到的、受 SDE 启发的采样器称为「SDE」变体，并指出它主要捕捉的是逐步骤的随机行为。我们实验对比了 ODE 形式化与该 SDE 变体。

3.3 条件与引导

控制模型生成是生成建模的重要方面。在图像扩散模型中，无分类器引导（CFG）[29] 已被确立为一种引导生成输出的、非常有效的技术。^{^1} CFG 还能够在生成质量与多样性之间进行权衡。由于 CFG 最初是为连续量（如分数函数或速度场）提出的，它天然适用于 ELF。这与离散情形形成对比——在离散情形中，CFG 在很大程度上仍未被探索，并且已被证明效果较差 [35, 57]。

在缺少类别标签的情况下，我们采用自条件 [10] 来构造 CFG 所需的条件信号。鉴于自条件已是 DLM 中的标准组件 [88, 15, 75, 46, 49, 67, 68]，引入 CFG 只会带来边际的额外计算开销。下面我们先描述 ELF 中使用的自条件，再介绍 CFG。

自条件。

在标准流匹配模型（即没有自条件）中，给定时间步的一次前向传播会产生单一预测。在我们的情形中将其记为 $\hat{x}'$ ，表示它是对干净嵌入 x 的预测。训练时，自条件 [10] 执行第二次前向传播，以 $\hat{x}'$ 为条件，它充当一个中间预测。第二次传播的输出记为 $\hat{x}$ ，可写为 $\hat{x} = \text{net}_\theta(z_t | \hat{x}', t)$ 。这一点通过把 $[z_t, \hat{x}]$ 拼接为网络输入来实现 [10]。训练时，模型以 50% 概率以 $\hat{x}'$ 为条件，否则使用空条件 0（详见附录图 10）。推理时，模型以上一时间步的预测为条件，因此推理时不引入额外的前向传播。

中间预测 $\hat{x}'$ 作为网络的条件。因此，它可以被当作接下来应用 CFG 时的条件信号 c 。

结合自条件的 CFG。

CFG [29] 通过线性外推组合无条件预测与条件预测。形式上，给定条件信号 c ，流匹配中的 CFG 定义速度场为 $v_{\text{cfg}}(z_t | c) = \omega v(z_t | c) + (1 - \omega) v(z_t | \emptyset)$ ，其中 $\emptyset$ 表示无条件对应项， ω 是引导尺度。如前所述，我们的条件信号 c 来自自条件。在其原始形式 [29] 中，CFG 在推理时被应用，每步需要两次前向传播。

为避免推理时的额外开销，我们采用了先前为图像生成而开发的「训练时 CFG」技术 [9, 78, 18, 19]。这些方法用单次网络传播来建模 v_{cfg} 而非 v （在我们的情形中是 x_{cfg} 而非 x ）。由于 ELF 的形式化与其图像生成对应物相似，把它适配到训练时 CFG 非常直接，这也进一步说明了我们基于连续形式化的优势。实现细节遵循 [18, 19] 中的形式，见附录（算法 3、4、5）。

扩展到条件生成。

至此，我们一直像先前工作 [63, 64, 35, 11] 那样在无条件的生成设定下介绍本方法。我们的方法可以自然地扩展到条件生成，即输出以某个输入序列（如提示）为条件。在该设定下，我们把条件序列的干净嵌入前置到模型输入前，并在训练与推理中保持它们不被扰动。模型随后可以通过自注意力机制以它们为条件。

CFG 在条件设定下仍然适用。此时条件 c 既包含自条件，也包含前置的干净嵌入；无条件对应项通过把 c 置零得到。类似于文生图生成 [16]，CFG 在我们的场景中能有效控制生成质量，而这可以被视为「文生文」生成。

^{^1} CFG 历史上是为类别条件生成而引入的。不过，「条件」这一概念可以推广到其他输入，例如文本提示。我们在这种更广泛的意义上使用 CFG，因为我们的设定不涉及类别标签。

4 实验

数据集与评测。

对于无条件生成，我们沿用此先前工作 [63, 64, 35, 11] 所使用的实验设计。我们在 OpenWebText (OWT) 数据集 [20] 上训练，它约有 9B 词元，并把序列打包成长度 $L = 1024$ 。评测时，我们生成 1,000 个样本，并报告生成困惑度 (Gen. PPL)，即生成样本在预训练 GPT-2 Large 模型 [59] 下的困惑度；同时报告平均 unigram 熵，作为样本多样性的度量。^{^2}

对于条件生成，我们考虑机器翻译与摘要。机器翻译使用 WMT14 德语→英语 (De-En) 数据集 [8]，序列长度 $L = 128$ (条件长度 64，目标长度 64；总计 144M 目标词元)，用 BLEU [55] 评测。摘要使用 XSum 数据集 [52]，序列长度 $L = 1088$ (条件长度 1024，目标长度 64；总计 6M 目标词元)，报告 ROUGE-1 (R1)、ROUGE-2 (R2) 与 ROUGE-L (R-L) [40]。我们把两者都视为序列到序列任务，条件生成不使用序列打包。

模型。

我们使用来自冻结的预训练 T5-small 编码器 [60] (35M) 的上下文嵌入，嵌入维度为 512。我们采用一种瓶颈设计，把嵌入线性投影到一个更低维度 (128) 空间，再将其投影回模型的隐藏维度 [37]。我们考虑三种模型规模：ELF-B (105M)、ELF-M (342M) 与 ELF-L (652M)，并以 ELF-B 作为消融的默认模型。详细配置见附录表 6。

训练与推理。

我们使用 Muon 优化器 [33] 训练模型，学习率为 0.002，batch 大小为 512。模型在 OWT 上训练 5 个 epoch (约 95K 步)，在 WMT14 与 XSum 上分别训练 100 个 epoch (约 880K 与 40K 步)。根据所选模型模式，网络要么用式 (1) 中的 MSE 损失 (80%)，要么用式 (2) 中的 CE 损失 (20%) 训练。推理时，我们使用 ODE 或 SDE 采样器生成样本。

^{^2} 我们不使用验证困惑度，因为对基于流的模型做似然评测可能需要额外的、与似然相关的训练 [1]。

4.1 消融实验

我们首先在 OWT 上的无条件生成这一更简单设定中，对模型的几个关键设计选择做消融，默认使用 ELF-B 模型与 64 步 ODE 欧拉采样器 (除非另有说明)。更多消融见附录 D。

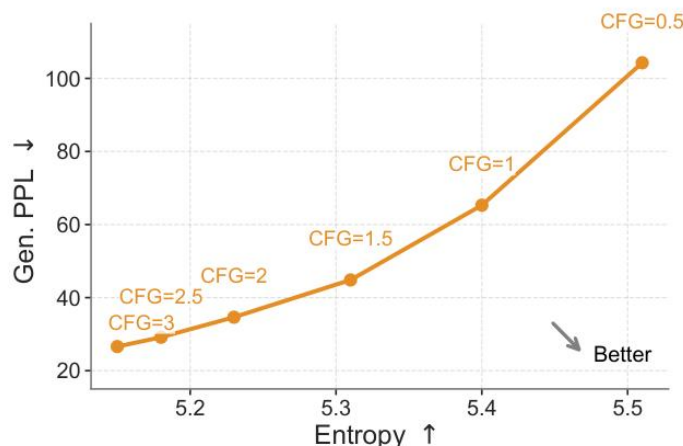


图 4：关于引导的消融。我们在不同 CFG 尺度下评估 Gen. PPL-熵的权衡：增大尺度会降低 Gen. PPL，但会减小熵。

无分类器引导（CFG）。

我们基于流的连续形式化天然兼容 CFG，而 CFG 是标准扩散模型中的一项极有效技术。因此，我们首先研究 CFG 尺度的效果。如图 4 所示，增大 CFG 尺度会降低生成困惑度，但也会减小熵，反映了质量-多样性的权衡。偏好的方向是图中右下区域，对应更低的生成困惑度与更高的熵。在其余大多数消融中，我们通过扫 CFG 尺度来评估这一质量-多样性权衡。曲线上的每个点都是在特定 CFG 尺度下，从 1,000 个生成样本计算得到的。

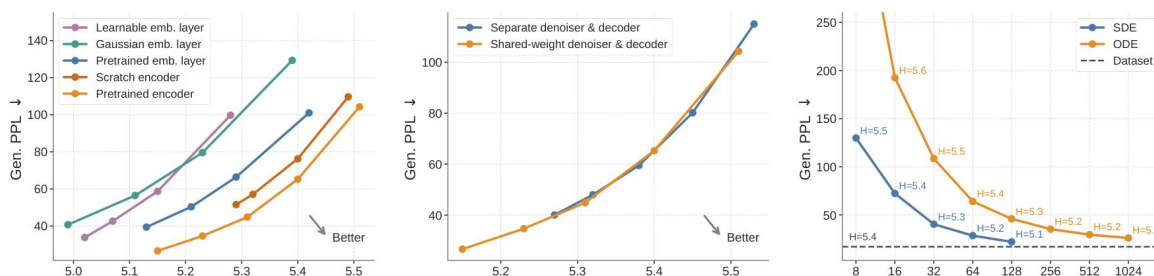


图 5：关于关键设计选择的消融。(a) 嵌入选择：我们对比上下文嵌入与非上下文嵌入，以及冻结与可学习嵌入；预训练的上下文嵌入取得最佳权衡。(b) 解码策略：我们对比共享权重去噪-解码器与两阶段、单独训练的解码器。两种策略取得相近的权衡，但共享权重变体能进一步延伸到低生成困惑度区域。(c) 采样器：我们对比 ODE 与 SDE 启发的采样器在不同采样步数下的表现；SDE 启发的采样器一致地在更少步数下达到更低的生成困惑度。

嵌入选择。

由于 ELF 运行在连续嵌入空间中，我们接下来研究嵌入选择如何影响性能。我们沿两个轴消融连续嵌入：嵌入是否是上下文的（即来自编码器）还是非上下文的（即来自单个嵌入层），以及它们是固定的还是可学习的。对于上下文嵌入，我们评测来自现成 T5 编码器 [60] 的嵌入，以及用原始 T5 目标在 OWT 上从零训练所得编码器的嵌入。对于非上下文嵌入，我们考虑预训练 T5 模型的词元嵌入、冻结的高斯嵌入以及可学习嵌入。详细设定见附录 E.3。结果见图 5a。上下文嵌入取得更好的生成困惑度-熵权衡。从零在 OWT 上训练的编码器嵌入表现良好，但略逊于预训练编码器的嵌入。在非上下文变体中，预训练词元嵌入优于冻结的高斯嵌入。可学习嵌入表现最差，可能是因为联合优化嵌入与去噪器很困难。总体而言，这些结果表明预训练上下文嵌入是 ELF 理想的语言表示。

解码策略。

由于我们用上下文嵌入作为连续表示，我们需要把它们解码回离散词元。我们使用共享权重网络，训练时交替进行 L_{MSE} 与 L_{CE} 。另外，我们还探索一种两阶段策略。第一阶段，用冻结的预训练 T5 编码器从零训练一个解码器，用 L_{CE} 从被掩码与带噪的嵌入中重建词元。第二阶段，冻结编码器与解码器，用 L_{MSE} 训练一个单独的去噪器（详见附录 E.3）。如图 5b 所示，两种策略取得相近的权衡，但共享权重变体进一步延伸到低生成困惑度区域，同时省去了额外的训练阶段，简化了流程。

采样器。

由于 ELF 在连续时间与连续空间中形式化，它天然支持确定性的 ODE 采样与随机的、类似 SDE 的采样；详见附录算法 6。我们以自条件 CFG 尺度为 1，对比不同采样预算下的 ODE 与 SDE 采样器。如图 5c 所示，SDE 采样在少步数区域内取得的生成困惑度显著低于 ODE 采样。这些结果表明，在采样时引入随机性可以有效减少误差累积，并提供更好的质量-效率权衡。

模型规模。

我们研究 ELF 在三种模型规模下的扩展行为：ELF-B（105M）、ELF-M（342M）与 ELF-L（652M）（详见附录表 6）。我们用 ODE 与 SDE 采样分别评测每个模型。如图 6 所示，扩展规模一致地改善了生成困惑度-熵边界。特别地，在熵匹配时，更大的模型取得更低的生成困惑度，表明在可比多样性下具有更高的样本质量。反之，在相似的生成困惑度下，更大模型保持更高的熵。采样器的影响在各模型规模下是一致的：SDE 采样通过把边界推向更优方向来优于 ODE 采样。这些结果表明 ELF 能有效扩展，展示了模型规模化的潜力。详细数值见附录表 10。

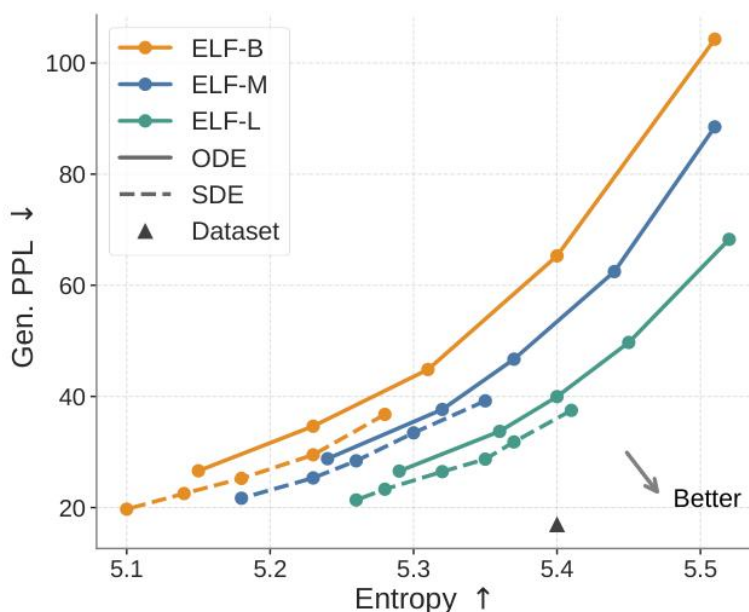


图 6：ELF 模型的规模化。我们对比 ELF-B、ELF-M 与 ELF-L。增大模型规模一致地改善 Gen. PPL-熵边界。

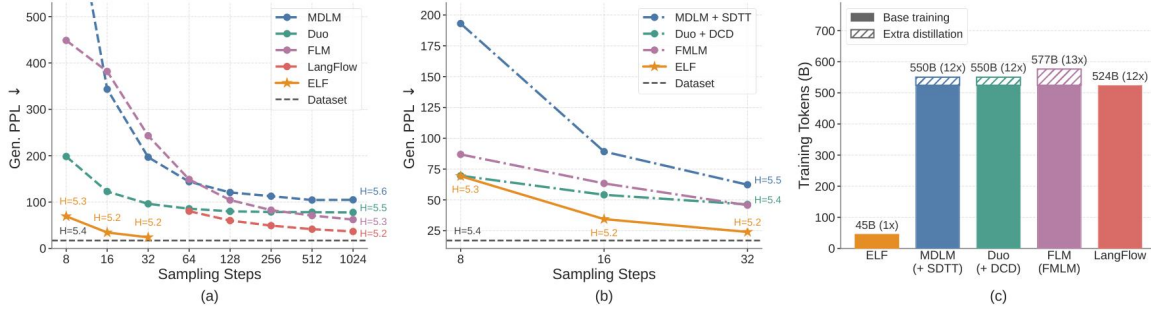


图 7: 系统级对比。ELF-B 优于在相似设定下训练的离散与连续 DLM (a)，也优于需要额外训练轮次的、其他基线的蒸馏变体 (b)，同时使用了少得多的训练词元 (c)。

4.2 无条件生成上的系统级对比

我们首先在可比设定下，把 ELF-B 与离散 DLM（包括 MDLM [63] 和 Duo [64]）以及连续 DLM（包括 FLM [35] 和 LangFlow [11]）进行对比。所有模型都在 OWT 数据集上训练。ELF 有 105M 参数，而对比基线约 170M 参数。对于 ELF，我们使用最佳配置：自条件 CFG 尺度为 3 的 SDE 采样（详见附录 E.2）。结果见图 7a。ELF 仅用 32 个采样步就达到了 24 的生成困惑度，所需的推理时计算比先前方法少得多。

即便与蒸馏模型相比，ELF 依然强劲：蒸馏模型需要额外训练来为少步生成蒸馏出一个学生模型。如图 7b 所示，在少步数区域内，ELF 优于蒸馏模型，包括 MDLM+SDTT [63, 13]、Duo+DCD [64] 与 FMLM [35]，而且完全不需要额外的蒸馏。我们进一步在附录 B 中探索 ELF 的渐进式蒸馏。蒸馏后的 ELF 模型在 1–32 个采样步内都优于蒸馏基线。

如图 7c 所示，就估算的训练词元而言，ELF 也显著更省数据。先前的 DLM 通常使用超过 500B 词元，而 ELF 只用 45B。³ 综合来看，这些结果表明，当与适当的采样与引导结合时，ELF 能达到很强的系统级性能。它不仅提升了推理效率，还在小得多的训练预算下取得强劲性能，展示了我们这种基于流的语言模型的潜力。ELF-B 生成的定性示例见图 8。

³ 各方法训练词元数的逐一分解见附录表 8。我们还尝试了用更多词元训练，但没有观察到性能进一步提升。

Model	Size	De-En [†]		XSum [‡]	
		BLEU ↑	ROUGE-1 ↑	ROUGE-2 ↑	ROUGE-L ↑
AR	99M	25.2	30.5 ± 0.13	10.2 ± 0.11	24.4 ± 0.12
MDLM [63]	99M	18.4	33.4 ± 0.11	11.6 ± 0.10	25.8 ± 0.10
Duo [64]	170M (+35M)	21.3 [‡]	31.4 ± 0.12	10.1 ± 0.10	25.0 ± 0.12
E2D2 [4]	99M	24.8	28.4 ± 0.11	8.3 ± 0.09	22.0 ± 0.10
SeqDiffuSeq [88]	-	21.3	19.3 [†]	1.7 [†]	14.1 [†]
CDCD [15]	-	24.9	-	-	-
Ours	105M (+35M)	26.4	36.0 ± 0.13	12.2 ± 0.11	27.8 ± 0.12

表 1: 机器翻译与摘要结果。我们在 WMT14 德→英 (De-En) 翻译与 XSum 摘要上评测 ELF-B，并与相似参数量级的基线对比。[†] 表示直接取自先前工作的结果，是 De-En 的默认来源；[‡] 表示我们用公开代码库复现的结果，是 XSum 的默认来源。对 XSum，在可用时我们额外给出跨评测样本的标准误。ELF 在两个设定下都取得最佳性能。

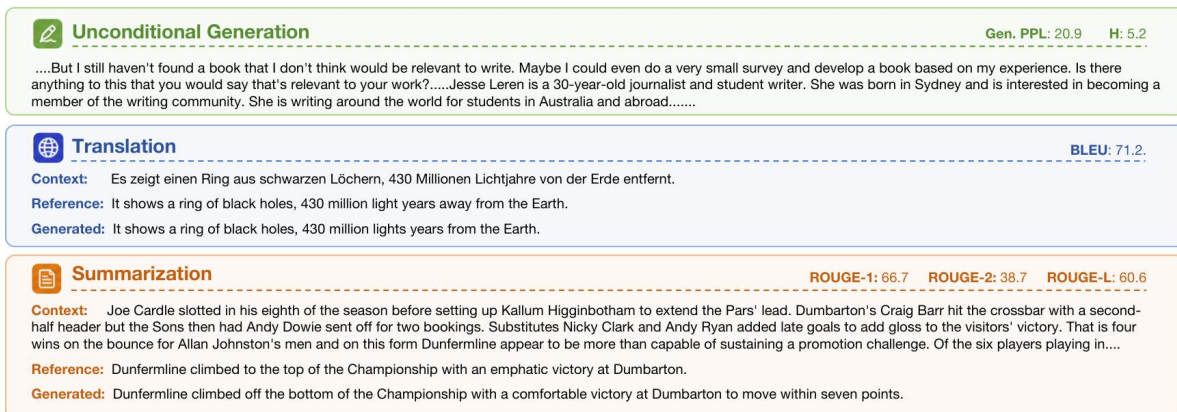


图 8：ELF-B 生成文本的定性示例。我们给出一个无条件样本、一个德→英翻译示例与一个摘要示例，并附上它们的自动评测指标。由于篇幅限制，部分文本被省略；更多示例见附录 F。

4.3 条件生成上的系统级对比

我们在相似的模型规模下，把 ELF-B 与自回归（AR）和基于扩散的基线进行对比。这些基线包括离散 DLM（MDLM [63]、Duo [64]、E2D2 [4]）与连续 DLM（SeqDiffuSeq [88] 和 CDCD [15]）。部分结果取自文献，部分我们用公开代码库复现。汇总见附录表 11。我们使用在验证集上选出的最佳采样配置：64 步 ODE 采样器，自条件 CFG 尺度设为 1，输入条件 CFG 尺度设为 2。

结果显示在表 1 中。ELF-B 在两个任务上都取得了所有对比方法中的最佳性能，展示了 ELF 在条件生成上的有效性。图 8 中的定性示例进一步表明，ELF-B 通常能遵循输入上下文，并生成与真值参考在语义上对齐的输出。

5 结论

我们提出了「嵌入式语言流」（ELF）——一种连续扩散语言模型，使用连续时间流匹配，在连续嵌入空间中形式化语言生成。与先前的 DLM 相比，ELF 保持去噪轨迹连续，只在最后一步施加离散化，从而能够直接迁移连续扩散模型的技术。实验上，与领先的离散 DLM 以及已有的连续 DLM 相比，ELF 在各项语言生成任务上都取得了很强的质量-效率权衡：以更少的采样步数与更少的训练词元达到更低的生成困惑度。这些结果表明，连续 DLM 仍是基于扩散的语言建模的一条有前景的方向。

致谢与资助披露

我们感谢 Mingyang Deng、Zhengyang Geng、Belinda Li、Itamar Pres 与 Laura Ruis 提供的宝贵反馈与深刻讨论。我们感谢 Google TPU 研究云（TRC）为我们提供 TPU 访问权限。本工作部分得到了 Siegel Family Foundation 的 Quest for Intelligence 项目支持。

参考文献

（注：参考文献按学术惯例保留原文，不进行翻译。）

- [1] Xinyue Ai, Yutong He, Albert Gu, Ruslan Salakhutdinov, J Zico Kolter, Nicholas Matthew Boffi, and Max Simchowitz. Joint distillation for fast likelihood evaluation and sampling in flow-based models. In ICLR, 2026.
- [2] Michael Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. JMLR, 2025.

- [3] Michael Samuel Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In ICLR, 2023.
- [4] Marianne Arriola, Yair Schiff, Hao Phung, Aaron Gokaslan, and Volodymyr Kuleshov. Encoder-decoder diffusion language models for efficient training and inference. In NeurIPS, 2025.
- [5] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. In NeurIPS, 2021.
- [6] Georgios Batzolis, Mark Girolami, and Luca Ambrogioni. Towards closing the autoregressive gap in language modeling via entropy-gated continuous bitstream diffusion. arXiv preprint arXiv:2605.07013, 2026.
- [7] Black Forest Labs, Stephen Batifol, Andreas Blattmann, Frederic Boesel, Saksham Consul, Cyril Diagne, Tim Dockhorn, Jack English, Zion English, Patrick Esser, Sumith Kulal, Kyle Lacey, Yam Levi, Cheng Li, Dominik Lorenz, Jonas Müller, Dustin Podell, Robin Rombach, Harry Saini, Axel Sauer, and Luke Smith. FLUX.1 Kontext: Flow matching for in-context image generation and editing in latent space. arXiv preprint arXiv:2506.15742, 2025.
- [8] Ondrej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Ales Tamchyna. Findings of the 2014 workshop on statistical machine translation. In ACL Workshop on Statistical Machine Translation, 2014.
- [9] Huayu Chen, Kai Jiang, Kaiwen Zheng, Jianfei Chen, Hang Su, and Jun Zhu. Visual generation without guidance. In ICML, 2025.
- [10] Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. In ICLR, 2023.
- [11] Yuxin Chen, Chumeng Liang, Hangke Sui, Ruihan Guo, Chaoran Cheng, Jiaxuan You, and Ge Liu. Langflow: Continuous diffusion rivals discrete in language modeling. arXiv preprint arXiv:2604.11748, 2026.
- [12] Oscar Davis, Samuel Kessler, Mircea Petrache, Ismail Ilkan Ceylan, Michael Bronstein, and Avishek Joey Bose. Fisher flow matching for generative modeling over discrete data. In NeurIPS, 2024.
- [13] Justin Deschenaux and Caglar Gulcehre. Beyond autoregression: Fast LLMs via self-distillation through time. In ICLR, 2025.
- [14] Justin Deschenaux, Caglar Gulcehre, and Subham Sekhar Sahoo. The diffusion duality, chapter ii: ψ -samplers and efficient curriculum. In ICLR, 2026.
- [15] Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, Curtis Hawthorne, Rémi Leblond, Will Grathwohl, and Jonas Adler. Continuous diffusion for categorical data. arXiv preprint arXiv:2211.15089, 2022.
- [16] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow Transformers for high-resolution image synthesis. In ICML, 2024.
- [17] Zhujin Gao, Junliang Guo, Xu Tan, Yongxin Zhu, Fang Zhang, Jiang Bian, and Linli Xu. Empowering diffusion models on the embedding space for text generation. In NAACL, 2024.
- [18] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. In NeurIPS, 2025.
- [19] Zhengyang Geng, Yiyang Lu, Zongze Wu, Eli Shechtman, J Zico Kolter, and Kaiming He. Improved mean flows: On the challenges of fast forward generative models. arXiv preprint arXiv:2512.02012, 2025.
- [20] Aaron Gokaslan and Vanya Cohen. Openwebtext corpus, 2019.
- [21] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and LingPeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models. In ICLR, 2023.
- [22] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. DiffuSeq-v2: Bridging discrete and continuous text spaces for accelerated seq2seq diffusion models. In Findings of EMNLP, 2023.
- [23] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation. In ICLR, 2026.
- [24] Ishaan Gulrajani and Tatsunori B Hashimoto. Likelihood-based diffusion language models. In NeurIPS, 2023.
- [25] Hongcan Guo, Qinyu Zhao, Yian Zhao, Shen Nie, Rui Zhu, Qiushan Guo, Feng Wang, Tao Yang, Hengshuang Zhao, Guoqiang Wei, and Yan Zeng. Continuous latent diffusion language model. arXiv preprint arXiv:2605.06548, 2026.

- [26] Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. SSD-LM: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In ACL, 2023.
- [27] Zhengfu He, Tianxiang Sun, Qiong Tang, Kuanning Wang, Xuan-Jing Huang, and Xipeng Qiu. Diffusionbert: Improving generative masked language models with diffusion models. In ACL, 2023.
- [28] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for Transformers. In Findings of EMNLP, 2020.
- [29] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In NeurIPS Workshops, 2021.
- [30] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In NeurIPS, 2020.
- [31] Vincent Tao Hu, Di Wu, Yuki M Asano, Pascal Mettes, Basura Fernando, Björn Ommer, and Cees G M Snoek. Flow matching for conditional text generation in a few sampling steps. In EACL, 2024.
- [32] Jaehyeong Jo and Sung Ju Hwang. Continuous diffusion model for language modeling. In NeurIPS, 2025.
- [33] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cecista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. Technical report, Keller Jordan blog, 2024.
- [34] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In NeurIPS, 2022.
- [35] Chanhuk Lee, Jaehoon Yoo, Manan Agarwal, Sheel Shah, Jerry Huang, Aditi Raghunathan, Seunghoon Hong, Nicholas M Boffi, and Jinwoo Kim. Flow map language models: One-step language modeling via continuous denoising. arXiv preprint arXiv:2602.16813, 2026.
- [36] Lijiang Li, Zuwei Long, Yunhang Shen, Heting Gao, Haoyu Cao, Xing Sun, Caifeng Shan, Ran He, and Chaoyou Fu. Omni-diffusion: Unified multimodal understanding and generation with masked discrete diffusion. arXiv preprint arXiv:2603.06577, 2026.
- [37] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. arXiv preprint arXiv:2511.13720, 2025.
- [38] Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A survey on diffusion language models. arXiv preprint arXiv:2508.10875, 2025.
- [39] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-LM improves controllable text generation. In NeurIPS, 2022.
- [40] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In ACL Workshop on Text Summarization Branches Out, 2004.
- [41] Zhenghao Lin, Yeyun Gong, Yelong Shen, Tong Wu, Zhihao Fan, Chen Lin, Nan Duan, and Weizhu Chen. Text generation with diffusion language models: A pre-training approach with continuous paragraph denoise. In ICML, 2023.
- [42] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In ICLR, 2023.
- [43] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In ICLR, 2023.
- [44] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In ICLR, 2019.
- [45] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In ICML, 2024.
- [46] Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q Weinberger. Latent diffusion for language generation. In NeurIPS, 2023.
- [47] Justin Lovelace, Varsha Kishore, Yiwei Chen, and Kilian Q Weinberger. Diffusion guided language modeling. In Findings of ACL, 2024.
- [48] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. SiT: Exploring flow and diffusion-based generative models with scalable interpolant Transformers. In ECCV, 2024.
- [49] Rabeeh Karimi Mahabadi, Hamish Ivison, Jaesung Tae, James Henderson, Iz Beltagy, Matthew E Peters, and Arman Cohan. Tess: Text-to-text self-conditioned simplex diffusion. In EACL, 2024.
- [50] Viacheslav Meshchaninov, Egor Chibulatov, Alexander Shabalin, Aleksandr Abramov, and Dmitry Vetrov. Cosmos: Compressed and smooth latent space for text diffusion modeling. In NeurIPS, 2025.

- [51] Viacheslav Meshchaninov, Alexander Shabalin, Egor Chibulatov, Nikita Gushchin, Ilya Koziev, Alexander Korotin, and Dmitry Vetrov. How to train your latent diffusion language model jointly with the latent space. arXiv preprint arXiv:2605.07933, 2026.
- [52] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. In EMNLP, 2018.
- [53] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In ICML, 2021.
- [54] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In NeurIPS, 2025.
- [55] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In ACL, 2002.
- [56] William Peebles and Saining Xie. Scalable diffusion models with Transformers. In ICCV, 2023.
- [57] Peter Potaptchik, Jason Yim, Adhi Saravanan, Peter Holderrieth, Eric Vanden-Eijnden, and Michael S Albergo. Discrete flow maps. arXiv preprint arXiv:2604.09784, 2026.
- [58] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. CANDI: Hybrid discrete-continuous diffusion models. arXiv preprint arXiv:2510.22510, 2025.
- [59] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI blog, 2019.
- [60] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR, 2020.
- [61] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In CVPR, 2022.
- [62] Daan Roos, Oscar Davis, Floor Eijkelboom, Michael Bronstein, Max Welling, Ismail Ilkan Ceylan, Luca Ambrogioni, and Jan-Willem van de Meent. Categorical flow maps. arXiv preprint arXiv:2602.12233, 2026.
- [63] Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In NeurIPS, 2024.
- [64] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin Chiu, and Volodymyr Kuleshov. The diffusion duality. In ICML, 2025.
- [65] Subham Sekhar Sahoo, Jean-Marie Lemerrier, Zhihan Yang, Justin Deschenaux, Jingyu Liu, John Thickstun, and Ante Jukic. Scaling beyond masked diffusion language models. arXiv preprint arXiv:2602.15014, 2026.
- [66] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In ICLR, 2022.
- [67] Alexander Shabalin, Viacheslav Meshchaninov, Egor Chibulatov, Vladislav Lapikov, Roman Kim, Grigory Bartosh, Dmitry Molchanov, Sergey Markov, and Dmitry Vetrov. TEncDM: Understanding the properties of the diffusion model in the space of language model encodings. In AAAI, 2025.
- [68] Alexander Shabalin, Simon Elistratov, Viacheslav Meshchaninov, Ildus Sadrtidinov, and Dmitry Vetrov. Why gaussian diffusion models fail on discrete data? arXiv preprint arXiv:2604.02028, 2026.
- [69] Noam Shazeer. GLU variants improve Transformer. arXiv preprint arXiv:2002.05202, 2020.
- [70] Junzhe Shen, Jieru Zhao, Ziwei He, and Zhouhan Lin. Codar: Continuous diffusion language models are more powerful than you think. arXiv preprint arXiv:2603.02547, 2026.
- [71] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In NeurIPS, 2024.
- [72] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In ICML, 2015.
- [73] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In ICLR, 2021.
- [74] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, Yuwei Fu, Jing Su, Ge Zhang, Wenhao Huang, Mingxuan Wang, Lin Yan, Xiaoying Jia, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Yonghui Wu, and Hao Zhou. Seed diffusion: A large-scale diffusion language model with high-speed inference. arXiv preprint arXiv:2508.02193, 2025.

- [75] Robin Strudel, Corentin Tallec, Florent Althé, Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, and Rémi Leblond. Self-conditioned embedding diffusion for text generation. arXiv preprint arXiv:2211.04236, 2022.
- [76] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [77] Jaesung Tae, Hamish Ivison, Sachin Kumar, and Arman Cohan. Tess 2: A large-scale generalist diffusion language model. In *ACL*, 2025.
- [78] Zhicong Tang, Jianmin Bao, Dong Chen, and Baining Guo. Diffusion models without classifier-free guidance. arXiv preprint arXiv:2502.12154, 2025.
- [79] Wan Team, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Fei Wu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. arXiv preprint arXiv:2503.20314, 2025.
- [80] Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. In *NeurIPS*, 2025.
- [81] Renzhi Wang, Jing Li, and Piji Li. InfoDiffusion: Information entropy aware diffusion process for non-autoregressive text generation. In *Findings of EMNLP*, 2023.
- [82] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kvcache and parallel decoding. In *ICLR*, 2026.
- [83] Tong Wu, Zhihao Fan, Xiao Liu, Hai-Tao Zheng, Yeyun Gong, Jian Jiao, Juntao Li, Jian Guo, Nan Duan, and Weizhu Chen. AR-Diffusion: Auto-regressive diffusion model for text generation. In *NeurIPS*, 2023.
- [84] Ling Yang, Ye Tian, Bowen Li, Xincheng Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. Mmada: Multimodal large diffusion language models. In *NeurIPS*, 2025.
- [85] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. arXiv preprint arXiv:2508.15487, 2025.
- [86] Jiasheng Ye, Zaixiang Zheng, Yu Bao, Lihua Qian, and Mingxuan Wang. DINOISER: Diffused conditional sequence learning by manipulating noises. *Transactions of the Association for Computational Linguistics*, 2024.
- [87] Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. Llada-v: Large language diffusion models with visual instruction tuning. arXiv preprint arXiv:2505.16933, 2025.
- [88] Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Fei Huang, and Songfang Huang. Seqdiffuseq: Text diffusion with encoder-decoder transformers. In *NAACL*, 2024.
- [89] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.
- [90] Yizhe Zhang, Jiatao Gu, Zhuofeng Wu, Shuangfei Zhai, Josh Susskind, and Navdeep Jaitly. PLANNER: Generating diversified paragraphs via latent language diffusion model. In *NeurIPS*, 2023.
- [91] Huangjie Zheng, Shansan Gong, Ruixiang Zhang, Tianrong Chen, Jiatao Gu, Mingyuan Zhou, Navdeep Jaitly, and Yizhe Zhang. Continuously augmented discrete diffusion model for categorical generative modeling. arXiv preprint arXiv:2510.01329, 2025.
- [92] Cai Zhou, Chenxiao Yang, Yi Hu, Chenyu Wang, Chubin Zhang, Muhan Zhang, Lester Mackey, Tommi Jaakkola, Stephen Bates, and Dinghuai Zhang. Coevolutionary continuous discrete diffusion: Make your diffusion language model a latent reasoner. In *ICML*, 2026.

附录 A 连续扩散语言模型综述

综述细节。

我们在表 2 中给出详细综述。该综述按若干设计轴总结了具有代表性的连续扩散与流式语言模型，包括底层的扩散或流过程、进行去噪的连续状态、在训练或推理时中间去噪状态是否会被离散化，以及是否需要单独的解码器将潜在状态映射回文本。

特别地，「训练时逐步离散化」与「推理时逐步离散化」两列区分了中间离散化的两种不同用途。

「训练时逐步离散化」表示中间去噪状态在训练时被映射为词元预测，并用词元级目标（如交叉熵损失）监督。这提供了直接的词表级引导，但也把中间去噪状态与类别预测耦合在一起。「推理时逐步离散化」表示中间采样状态在生成时被显式投影回词元对齐的表示，例如嵌入空间中的最近邻取整，或单纯形上的 argmax 投影。没有推理时逐步离散化的方法会保持采样轨迹连续，只在最后一步做离散化。「单独解码器」列表示某方法是否需要单独的解码器（单独或与扩散模型联合训练）把连续潜在表示映射回离散文本。

Method	Process [†]	State [‡]	Train per-step discr.	Infer. per-step discr.	Sep. dec.
<i>Embedding-space Diffusion LMs</i>					
Diffusion-LM [Li et al., 2022]	DDPM	learn emb	Yes	Yes	
SED [Strudel et al., 2022]	DDPM	fix emb	Yes		
CDCD [Dieleman et al., 2022]	Score-ODE	learn emb	Yes		
DiffuSeq [Gong et al., 2023]	DDPM	learn emb	Yes	Yes	
GENIE [Lin et al., 2023]	DDPM	learn emb	Yes		
AR-Diffusion [Wu et al., 2023]*	DDPM	learn emb	Yes		
Plaid [Gulrajani and Hashimoto, 2023]	VLB	learn emb	Yes		
InfoDiffusion [Wang et al., 2023]	DDPM	learn emb	Yes		
Difformer [Gao et al., 2024]	DDPM	learn emb	Yes		
SeqDiffuSeq [Yuan et al., 2024]	DDPM	learn emb	Yes		
DINOISER [Ye et al., 2024]	SDE/DDIM	learn emb	Yes		
FlowSeq [Hu et al., 2024]	FM	learn emb	Yes		
<i>Simplex Diffusion LMs</i>					
SSD-LM [Han et al., 2023]*	DDPM	simplex	Yes	Yes	
TESS [Mahabadi et al., 2024]	DDPM	simplex	Yes	Yes	
RDLM [Jo and Hwang, 2025]	RDM	simplex	Yes		
TESS 2 [Tae et al., 2025]	DDPM	simplex	Yes	Yes	
Fisher-Flow [Davis et al., 2024]	FM	simplex			
<i>Latent Diffusion LMs</i>					
LD4LG [Lovelace et al., 2023]*	DDPM	fix enc			Yes
PLANNER [Zhang et al., 2023]*	DDPM	fix enc			Yes
DGLM [Lovelace et al., 2024]*	VP-DDPM	fix enc			Yes
TEncDM [Shabalin et al., 2025]	VP-DDPM	fix enc			Yes
Cosmos [Meshchaninov et al., 2025]	VP-DDPM	fix enc			Yes
CoDAR [Shen et al., 2026]*	VP-SDE	fix enc			Yes
LDLM [Meshchaninov et al., 2026]	VP-DDPM	learn enc			Yes
Cola DLM [Guo et al., 2026]*	FM	learn enc			Yes
<i>Flow-based LMs</i>					
CFM [Roos et al., 2026]	FM	simplex	Yes		
FLM [Lee et al., 2026]	FM	one-hot	Yes		
DFM [Potapchik et al., 2026]	FM	simplex	Yes		
LangFlow [Chen et al., 2026]	Bregman FM	learn emb	Yes		
ELF (ours)	FM	fix enc			

表 2: 连续扩散与流式语言模型综述。我们按若干设计轴总结了具有代表性的连续扩散与流式语言模型。「Process」表示扩散或流过程，绿色表示连续时间形式化，红色表示离散时间形式化。「State」表示进行去噪的连续状态。「训练时逐步离散化」标记那些在训练中把中间去噪状态转换为词元预测，并在中间步骤施加词元级监督（如交叉熵损失）的方

法。「推理时逐步离散化」标记那些在生成时把中间采样状态投影回词元对齐状态的方法。「单独解码器」标记那些需要单独解码器把潜在表示映射回文本的方法。空白项表示没有。* 表示自回归或块自回归生成。

ELF 的定位。

表 2 显示，已有的连续 DLM 在去噪过程定义于何处、以及连续状态如何映射回文本方面差异很大。许多嵌入空间与单纯形方法在中间去噪步骤通过词元级目标（通常为交叉熵）使用训练时逐步离散化。这些目标提供了直接的词元级引导，同时使去噪轨迹与词表级预测耦合得更紧。潜在扩散语言模型往往避免这种逐步词表监督，但通常依赖 DDPM 风格或基于分数的形式化与 DDPM 噪声调度，并且需要一个单独的潜在到文本解码器（如自回归解码器、非自回归解码器或潜在解压器）来恢复离散词元，该解码器单独或与扩散模型联合训练。

ELF 占据了不同的设计点。它在冻结的上下文嵌入空间中形式化连续时间流匹配，并保持采样轨迹连续，只在最后的解码步骤施加离散化。与先前的潜在扩散语言模型不同，ELF 不需要单独解码器：一个共享权重网络在中间步骤做去噪，并在最后步骤通过反嵌入层恢复词元。

附录 B ELF 面向少步语言生成的渐进式蒸馏

我们已经证明 ELF 在 8–32 个采样步下取得强劲性能，优于依赖蒸馏的先前离散与连续 DLM。然而，随着采样步数进一步减少，其性能会下降，一步/少步生成对 ELF 而言仍然困难。渐进式蒸馏已被证明能在保持生成质量的同时大幅减少采样步数 [66]。受此启发，我们提出了带渐进式蒸馏的 ELF (ELF+PD)：它把一个预训练的 ELF 教师蒸馏为一个学生模型，用于少步语言生成。

B.1 方法

渐进式蒸馏 [66] 的核心思想是把 K 个教师采样步压缩为单个学生步。给定时间区间 $[t, r]$ ，我们用数值求解器应用 K 个教师采样步，把带噪嵌入从 z_t 移到 z_r 。然后把得到的教师位移转换成一个新目标：

$$\begin{aligned} \text{target:} \quad & \tilde{x} = z_t + \frac{1-t}{r-t}(z_r - z_t). \\ \text{An ELF+PD student with parameters } \theta \text{ by minimizing} \end{aligned} \quad (式 3)$$

我们通过最小化以下目标来训练参数为 θ 的 ELF+PD 学生：

$$\mathcal{L}_{\text{distill}} = \mathbb{E}_{t,x,\epsilon} \left\| x_{\theta}(z_t, t) - \tilde{x} \right\|^2. \quad (式 4)$$

我们保留 ELF 的双分支训练设定，并使用相同的共享权重去噪器与解码器。对于去噪分支，我们把式 (1) 中的 MSE 损失替换为式 (4) 中的蒸馏损失。对于解码分支，我们保留式 (2) 中的原始交叉熵损失。

我们通过每一轮把步数减半，把教师渐进式蒸馏为一个单步学生，如 [66, 63, 64] 中那样。我们使用表 3 所示的课程，进行五轮蒸馏。五轮中都使用固定的 64 步 ELF 教师。第一个 16 步学生由教师初始化，而后续每个学生都由上一轮的学生初始化。第一轮之后，每一轮都把学生的步数减半。当目标是 N 步学生时，每个学生步匹配 $64/N$ 个教师子步，使学生能用更少的采样步近似 64 步教师轨迹。

Round	Student steps N	Teacher substeps
r_1	16	4
r_2	8	8
r_3	4	16
r_4	2	32
r_5	1	64

表 3: ELF+PD 蒸馏课程。每一轮把学生步数减半，同时保持固定的 64 步教师。一个 N 步学生的每步匹配 $64/N$ 个教师子步。

B.2 实验设定

我们使用 OpenWebText 数据集，并遵循第 4 节所述的实验设定。对于每轮蒸馏，我们使用与原始 ELF 训练相同的超参数设定（见表 7）训练一个 epoch，含 0.1 个 warmup epoch。与 ELF 类似，我们使用带自条件的训练时 CFG：为每个样本采样一个 CFG 尺度，教师与学生都以相同的自条件 CFG 尺度为条件。我们为学生与教师模型都使用相同的 logit-normal 调度来采样时间步。

推理时，我们使用与训练时相同的 logit-normal 时间调度的 SDE 启发采样器。对于 1、2、4、8 步生成，我们把噪声再注入尺度设为 $\gamma = 1.5$ ，自条件 CFG 尺度设为 2.5。对于 16 与 32 步生成，我们使用 $\gamma = 2.0$ 与自条件 CFG 尺度 2.0。

对于图 9c 中的自条件 CFG 扫描，我们把自条件 CFG 尺度在 0.5 到 3.0 之间扫描，同时固定 $\gamma = 1.5$ ，并且只报告熵大于 5.0 的配置，以排除退化的、重复的文本。为了研究表 5 中的课程蒸馏效果，我们固定 $\gamma = 1.5$ ，自条件 CFG 尺度为 2.5。

B.3 结果

少步生成。

图 9a 在 OpenWebText 上比较 ELF+PD 与具有代表性的蒸馏离散与连续 DLM 基线：离散模型的蒸馏版本（MDLM+SDTT [63, 13] 与 Duo+DCD [64]），以及连续的基于流的模型（FMLM [35]）。ELF+PD 在每个采样预算上都取得最低的生成困惑度，同时保持合理的熵。得益于 ELF 的数据效率，ELF+PD 也使用了少得多的训练词元，如图 9b 所示。精确数值在表 4 中报告。我们还在图 9c 中比较 ELF+PD 在不同采样步数下的生成困惑度-熵权衡。8-32 步之间的权衡相当，而使用少于 8 步会导致性能下降。

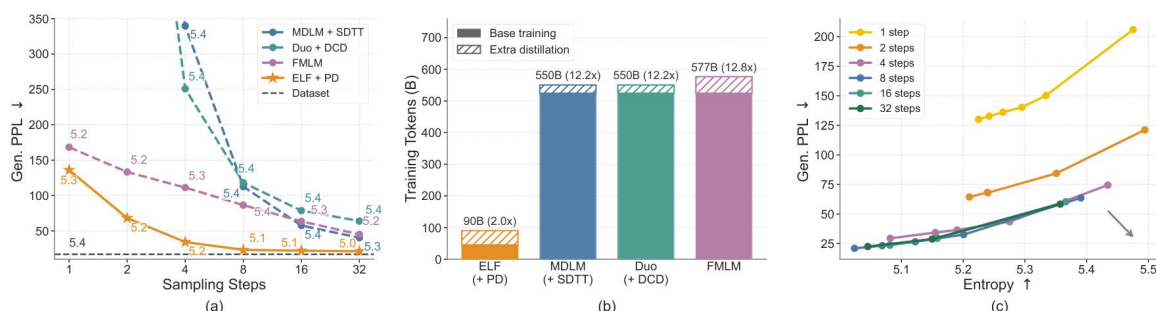


图 9: 带渐进式蒸馏的 ELF (ELF+PD) 与 OpenWebText 上的蒸馏 DLM 基线对比。(a) 生成困惑度随采样步数的变化: ELF+PD 在所有采样步数上都优于蒸馏基线。点的标签表示熵。(b) 估算的训练词元: ELF+PD 只使用 90B 词元（基础模型训练的 2.0 倍），而其他基线需要 550-577B（12 倍以上）训练词元。(c) 通过扫描自条件 CFG 尺度得到的、ELF+PD 在不同采样步数下的生成困惑度-熵权衡。8-32 步之间的权衡相当，而使用少于 8 步会导致性能下降。

Steps	MDLM + SDTT [63, 13]		Duo + DCD [64]		FMLM [35]		ELF + PD (Ours)	
	Gen. PPL ↓	Entropy ↑	Gen. PPL ↓	Entropy ↑	Gen. PPL ↓	Entropy ↑	Gen. PPL ↓	Entropy ↑
1	1260.60	5.26	5743.90	6.02	168.30	5.17	136.10	5.26
2	877.22	5.34	891.16	5.41	133.29	5.25	68.25	5.24
4	339.73	5.38	250.86	5.37	111.31	5.26	34.33	5.16
8	112.66	5.41	118.21	5.41	86.50	5.36	23.18	5.07
16	57.74	5.39	78.74	5.43	63.63	5.29	22.12	5.06
32	40.41	5.34	63.98	5.40	45.09	5.25	21.32	5.04

表 4: OpenWebText 上的少步无条件生成。我们比较五轮蒸馏后的 ELF+PD 与蒸馏的离散和连续扩散语言模型的基线。

蒸馏课程的效果。

如前述，我们使用五轮课程把教师渐进式蒸馏为单步学生。每一轮把学生步数减半，并把每个区间内的教师步数翻倍。表 5 给出了学生在课程各阶段的性能。早期轮次的模型只在较大的采样预算下表

现良好，在较小预算下会崩溃为退化输出；而后期轮次的模型大幅改善了 1–4 步生成，同时保持合理熵。最后一轮之后，单步学生在 1、2、4、8 步都取得最佳生成困惑度；定性示例见 F.4 节。

Round	step = 1		step = 2		step = 4		step = 8	
	Gen. PPL ↓	Entropy ↑	Gen. PPL ↓	Entropy ↑	Gen. PPL ↓	Entropy ↑	Gen. PPL ↓	Entropy ↑
r_1	4.9	1.72*	119.4	5.10	143.4	5.37	58.0	5.26
r_2	1.7	0.64*	171.1	5.24	128.6	5.40	36.6	5.24
r_3	30.7	3.60*	153.1	5.44	69.0	5.37	27.9	5.20
r_4	165.9	5.35	92.2	5.36	46.1	5.28	27.3	5.19
r_5	136.1	5.26	68.2	5.24	34.3	5.16	23.2	5.07

表 5: ELF 渐进式蒸馏各轮与各采样步的结果。早期轮次的学生在较小的采样预算下会崩溃，而后期轮次的学生大幅改善少步性能。* 表示退化结果，即熵低于 5.0。

附录 C 方法细节

C.1 训练

我们在图 10 中展示完整的训练流程。输入词元首先被编码为干净嵌入 $\mathbf{x}$ ，随后在被送入 ELF 模型之前经历三个关键步骤：扰动、自条件，以及为条件与引导添加控制词元。在去噪分支中，模型预测干净嵌入 $\hat{\mathbf{x}}$ ，并用 L_MSE 监督。在解码分支中，同一个共享权重网络预测嵌入，随后通过一个反嵌入层，并用 L_CE 监督。完整的训练算法见算法 3 与算法 4。

嵌入扰动。

首先，我们通过加噪声来扰动干净嵌入 $\mathbf{x}$ 。具体而言，我们使用 $\mathbf{z}_t = t\mathbf{x} + (1-t)\epsilon$ 获得带噪嵌入 $\mathbf{z}_t$ ，其中 ϵ 为高斯噪声， t 为时间步。在扰动之前，我们先使用从 OWT 数据集估计的均值与标准差来归一化干净嵌入。我们为不同模式使用不同的噪声调度。

对于去噪分支，我们为每个序列从 **logit-normal** 分布采样时间步 t 。具体地，我们抽取 $t' \sim N(P_mean, P_std^2)$ ，并通过 $t = \sigma(t')$ 把它映射到单位区间，其中 $\sigma(\cdot)$ 表示 **sigmoid** 函数。在所有实验中，我们使用 $P_mean = -1.5$ 与 $P_std = 0.8$ 。我们把高斯噪声按因子 2 重新缩放。

对于解码分支，我们通过把模型条件化为解码器模式（即 $t = 1$ ）来训练最后一步的离散化。在该时间步， $\mathbf{z}_t$ 对应干净嵌入。因此，为使最后一步输入非平凡化，我们用一个从不同噪声调度采样的逐词元扰动水平 p 扰动干净嵌入。具体地，我们从 $P_mean = 0.8$ 与 $P_std = 0.8$ 的 **logit-normal** 分布抽取 p ，并构造 $\tilde{\mathbf{z}} = p\mathbf{x} + (1-p)\epsilon$ ，把 ϵ 乘上一个噪声尺度。我们对 OWT 与条件生成任务分别使用 5 与 1 的噪声尺度。结果是，同一序列内各词元的扰动水平各不相同。这一设计鼓励共享权重的解码器模式从其周围上下文恢复被扰动的嵌入，使最后一步的离散化对推理时去噪器产生的不完美嵌入更加鲁棒。

自条件。

我们按照先前工作 [10] 施加自条件。训练时，以一定概率，我们执行一次额外的前向传播获得预测嵌入 $\hat{\mathbf{x}}'$ ，它沿通道维与带噪嵌入 $\mathbf{z}_t$ 拼接。我们让梯度穿过预测嵌入 $\hat{\mathbf{x}}'$ 时停止。对于其余样本，我们把 $\mathbf{z}_t$ 与全零嵌入 $\mathbf{0}$ 拼接。由于这种拼接使通道维翻倍，我们用线性层把它投影回原始维度。在去噪分支中，我们以 50% 概率用 $\hat{\mathbf{x}}'$ 施加自条件。对于解码分支，我们总是用 $\mathbf{0}$ 作为自条件输入，如算法 4 所示。

训练时 CFG。

如 3.3 节所述，我们的模型执行带自条件的训练时 CFG [18, 19, 9, 78]。在训练时 CFG 中，网络被设计为建模组合后的量 $\mathbf{v}_{cfg_0}$ ，而非组合前的量 $\mathbf{v}_0$ 。遵循 [18, 19]，回归目标 $\mathbf{v}_{target}$ 现在为：

$$\mathbf{v}_{target} = \mathbf{x} - \epsilon + \left(1 - \frac{1}{\omega}\right) \left(\mathbf{v}_{\theta}^{cfg}(\mathbf{z}_t | t, \mathbf{c}, \omega) - \mathbf{v}_{\theta}^{cfg}(\mathbf{z}_t | t, \emptyset, \omega)\right),$$

(式 5)

其中 ω 是引导尺度。当 $\omega = 1$ 时，上式退化为没有训练时 CFG 的情形。此时损失变成 $\|\mathbf{v}_{cfg_0}(\cdot) - \mathbf{v}_{target}\|^2$ [18, 19]。见算法 3。对每个训练样本，我们从偏向较小值的幂分布中随机采样一个自条件 CFG 尺度 $\omega \in [0.5, 5.0]$ [18, 19]。由于 ELF 使用 **x-prediction**，量 $\mathbf{v}$ 总是从其 $\mathbf{x}$ 预测对应项（条件的或无条件的）转换而来。

我们的模型使用各种各样、齐备的条件。标准扩散模型通常通过 adaLN-Zero [56] 实现条件，它通过求和把所有条件信号组合起来。当存在许多异构条件时，这种设计效果会变差。因此，我们采用上下文内条件 [19]，通过前置一组编码条件信息的控制词元来实现。每个控制词元嵌入的维数与标准语言词元嵌入相同。我们前置三类控制词元：4 个取值在 $[0, 1]$ 的时间词元、4 个从 $[0.5, 5]$ 采样的 CFG 尺度词元，以及 4 个指示去噪或解码的模型模式词元。这些词元与模型联合训练。所有连续值（即时间与 CFG 尺度）都用位置嵌入编码。

对于条件生成，我们把条件序列的干净嵌入放在控制词元之后、待生成目标序列之前。模型随后对条件与目标词元的拼接序列做双向自注意力。条件嵌入在训练期间保持不被扰动。为了对条件生成启用 CFG，我们以 10% 概率随机丢弃条件，即把条件序列的嵌入置零。这使得模型能够在同一框架下学习条件与无条件生成。

mode, i.e., $t = 1$. At this time step, z_t corresponds to clean embeddings. Therefore, to make the

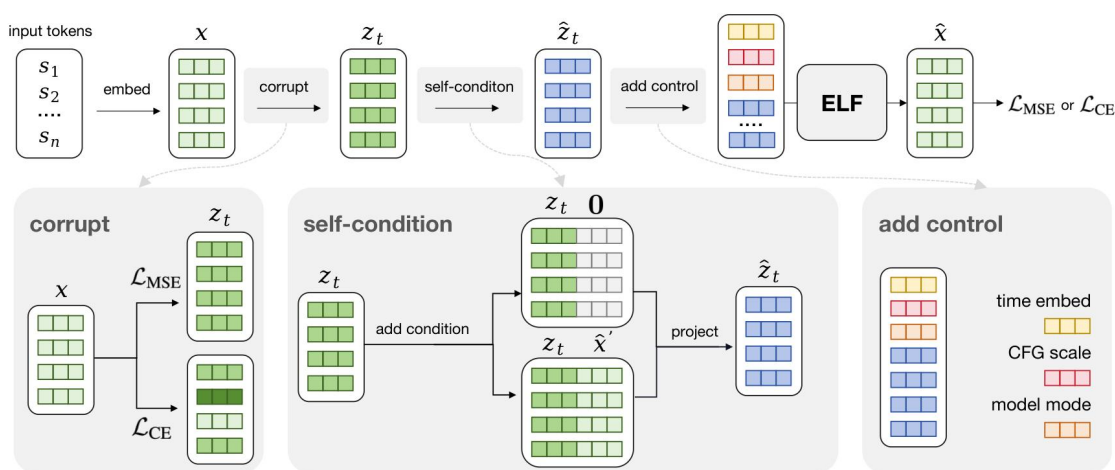


图 10: 我们训练流程的示意。从干净嵌入 x 出发，我们在两种模式下施加不同噪声调度以获得被扰动的嵌入 z_t 。然后通过沿通道维拼接 0 或上一步预测 $\hat{x}'$ 来施加自条件，并把拼接后的嵌入投影回原始维度以形成 $\hat{z}_t$ 。接下来，我们在嵌入序列前置控制词元，包括取值在 $[0, 1]$ 的时间词元、取值在 $[0.5, 5]$ 的 CFG 尺度词元，以及指示去噪或解码的模型模式词元。所得序列被送入 ELF 以产生最终预测 $\hat{x}$ ，它用去噪损失 $\mathcal{L}_{MSE}$ 或逐词元交叉熵损失 $\mathcal{L}_{CE}$ 监督。

Algorithm 3 ELF denoiser training with conditioning and guidance.

```
# net(z, t, c, w, mode): ELF network with in-context conditioning
# self_cond_proj(z): Self-conditioning projection layer that converts concatenated
#   embeddings back to the original embedding dimension
# self_cond_prob: Self-conditioning probability
# s: a sequence of discrete tokens
# c: condition (only for conditional generation)

x = encode(s)
t = sample_t()
w = sample_sc_cfg_scale()
e = randn_like(x)
z = t * x + (1 - t) * e
v = x - e

# z w/o self-conditioning
z_no_sc = self_cond_proj(concat([z, zeros_like(z)], dim=-1))
x_no_sc = net(z_no_sc, t, c, w, mode="denoise")
v_no_sc = (x_no_sc - z) / (1 - t)

# z w/ self-conditioning
z_sc = self_cond_proj(concat([z, stopgrad(x_no_sc)], dim=-1))
x_sc = net(z_sc, t, c, w, mode="denoise")
v_sc = (x_sc - z) / (1 - t)

# Compute CFG target
v_target = v + (1 - 1 / w) * (v_sc - v_no_sc)

# Apply per-example self-conditioning mask
self_cond_mask = uniform(x.shape[0]) < self_cond_prob
v_pred = where(self_cond_mask, v_sc, v_no_sc)
v_target = where(self_cond_mask, v_target, v)
v_target = stopgrad(v_target)

# Compute loss
```

算法 3: 带条件与引导的 ELF 去噪器训练。

Algorithm 4 ELF decoder training with conditioning and guidance.

```
# net(z, t, c, w, mode): ELF network with in-context conditioning
# self_cond_proj(z): Self-conditioning projection layer that converts concatenated
#   embeddings back to the original embedding dimension
# s: a sequence of discrete tokens
# c: condition (only for conditional generation)

x = encode(s)
p = sample_per_token_p()
w = sample_sc_cfg_scale()
e = randn_like(x)
z = p * x + (1 - p) * e

# use z w/o self-conditioning
```

算法 4: 带条件与引导的 ELF 解码器训练。

C.2 推理

我们在算法 5 中展示完整的推理算法。由于自条件 CFG 尺度通过上下文内条件给出，改变 w 不需要额外的推理传播。通过把 w 作为模型输入来修改，我们可以灵活地控制生成质量与多样性之间的权衡。

时间调度。

我们使用 logit-normal 时间调度，把连续时间区间 $t \in [0, 1]$ 离散为 T 个区间。具体地，我们从与训练时去噪分支相同的 logit-normal 分布中采样 $T - 1$ 个时间步，并把它们排序以构成中间点。我们使用 $P_mean = -1.5$ 、 $P_std = 0.8$ 以匹配训练时的 logit-normal 分布。我们确保第一个区间从 $t = 0$ 开始，最后一个区间在 $t = 1$ 结束。这种调度在 t 靠近 0 时产生更小的区间，在 t 靠近 1 时产生更大的区间。它表现出很强的实证性能，很可能是因为噪声更强的区域需要更细的离散化，而且这种调度更好地匹配训练时使用的噪声分布。

采样器。

我们的方法既支持确定性的 ODE 采样，也支持受 SDE 启发的随机采样器。算法 2 中的主算法为简单起见使用 ODE 采样器，而算法 6 总结了两种采样器的单步更新。

SDE 变体受到与流匹配相关联的 SDE 的启发 [48]，其动力学可被解释为在每一步注入无穷小噪声。实际中，我们采用一个简单近似：在每个采样步重新注入高斯噪声，同时把时间变量略微移向噪声更强的区间。我们引入噪声再注入尺度 γ 来控制每步加入的随机性大小。然后在被扰动的状态上评估去噪器，并用其干净嵌入预测来更新原始状态。当 $\gamma = 0$ 时，不施加随机扰动，更新退化为确定性的 ODE 采样。

条件生成的 CFG。

我们结合条件与无条件预测来施加标准 CFG。类似地，我们用 CFG 尺度来控制引导强度。

Algorithm 5 ELF inference with conditioning and guidance.

```
# net(z, t, c, w, mode): ELF network with in-context conditioning
# self_cond_proj(z): Self-conditioning projection layer that converts concatenated
#   embeddings back to the original embedding dimension
# shape: embeddings shape
# ts: discretized time grid over [0, 1] with N intervals
# c: condition (only for conditional generation)
# w: self-conditioning CFG scale

z = randn(shape)
x_pred = zeros(shape)

for i in range(len(ts) - 1):
    t = ts[i]
    dt = ts[i + 1] - ts[i]
    # Self-condition on the previous prediction
    z_sc = self_cond_proj(concat([z, x_pred], dim=-1))
    x_pred = net(z_sc, t, c, w, mode="denoise")
    # convert x prediction to velocity
    v = (x_pred - z) / (1 - t)
    z = z + dt * v
```

算法 5：带条件与引导的 ELF 推理。

Algorithm 6 ELF inference with different samplers.

```
# z: noisy embeddings of current time step
# t: current time step
# dt: time interval, t_next - t
# gamma: controls the amount of noise added back in the SDE sampler

def ode_step(z, t, dt):
    x_hat = net(z, t, mode="denoise")
    v = (x_hat - z) / (1 - t)
    z = z + dt * v
    return z

def sde_step(z, t, dt, gamma):
    # Re-inject noise and move back to the corresponding time step
    # The jump size is defined relative to the time-step interval
    e = randn_like(z)
    alpha = 1 - gamma * dt
    t_back = alpha * t
    z_back = alpha * z + (1 - alpha) * e

    x_hat = net(z_back, t_back, mode="denoise")
    v = (x_hat - z) / (1 - t)
    z = z + dt * v
    return z
```

算法 6: 使用不同采样器的 ELF 推理。

附录 D 更多消融实验

在本节中，我们给出关于设计选择的更多消融。除非另有说明，所有实验都使用 64 步 ODE 采样器，或 $\gamma = 1$ 的 64 步 SDE 采样器的时间调度。与之前一样，我们通过改变自条件 CFG 尺度来评估生成困惑度-熵权衡。我们用红色表示生成质量较差的区域，即熵低于 5.0（通常对应重复或退化的句子），或生成困惑度高于 300（通常对应语义无意义或不合语法的句子）。所有模型都训练相同的步数，其他配置都与默认设定一致。

D.1 预测目标

我们的模型直接预测干净嵌入 $\mathbf{x}$ (x-prediction)。这使我们能通过权重共享使用统一的去噪器与解码器，并用去噪目标 L_{MSE} 与词元级目标 L_{CE} 联合优化模型。先前的工作也表明 x-prediction 是必要的，因为高维干净数据往往位于低维流形上 [37]。

这里，我们进一步研究预测目标的效果。具体地，由于存在三个量与两个约束：线性插值 $\mathbf{z}_t = t\mathbf{x} + (1-t)\epsilon$ 与流动速度 $\mathbf{v} = \mathbf{x} - \epsilon$ ，网络可以被训练来预测其中一个量，即 $\mathbf{x}$ -、 $\mathbf{v}$ - 或 ϵ -prediction。为了在受控设定下研究这一点，我们用两阶段预训练编解码器设定：预训练 T5 编码器把词元映射为连续嵌入，并训练一个解码器从掩码与带噪嵌入中重建嵌入（详见 E.3 节）。我们只训练去噪模型，保持编码器与解码器固定。我们使用 adaLN-Zero 条件与 64 步 ODE 采样器来绘制生成困惑度-熵权衡曲线。

为了研究预测目标在嵌入维度增大时的表现，我们分别考虑 T5-small、T5-base 与 T5-large 编码器，对应嵌入维度 512、768 与 1024。我们将瓶颈维度设为对应的输入嵌入维度。如图 11 所示，x-prediction 在所有维度下都最稳定，即便在 1024 维也能保持合理的生成困惑度-熵权衡。相比之下，v-prediction 在 512 维有竞争力，但随着维度增大而退化，在 768 与 1024 维时生成困惑度显著更高。 ϵ -prediction 在所有维度下都崩溃，要么熵极低，要么生成困惑度极高，表明生成了重复、退化或不合语法的文本。这些结果支持了如下假设：干净数据预测更适合高维语言表示，这与先前工作的发现一致 [37]。

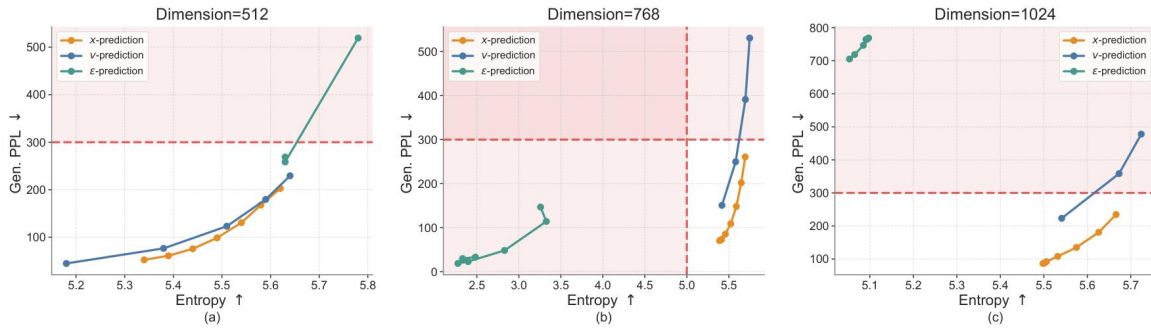


图 11：预测目标的效果。我们通过分别使用 T5-small、T5-base 与 T5-large 编码器，把输入维度从 512 调到 768 再到 1024。在所有输入维度下，x-prediction 都保持稳定且表现良好。相比之下，v-prediction 在 512 维表现良好但在高维退化，而 ϵ -prediction 在从 512 到 1024 的所有维度下都崩溃。红色区域表示质量较差的生成，其中熵低于 5（如重复句子）或生成困惑度超过 300（如无意义或不合语法的句子）。这与先前工作「高维干净数据通常位于低维流形上」的假设一致 [37]。

D.2 瓶颈

我们的模型使用瓶颈设计，把编码器表示投影到一个更低维空间，再映射回模型隐藏维度。这一设计源于如下假设：自然数据可能位于高维嵌入空间内的低维流形上。我们对比 32、128 与 512 三种瓶颈

维度，结果显示在图 12 中。瓶颈维度对生成困惑度-熵权衡有明显影响。在 ODE 采样下，三种瓶颈大小遵循相似边界，但较小的瓶颈往往以更低熵为代价达到更低的生成困惑度。在 SDE 采样下，差异变得更加显著：32 维瓶颈达到最低的生成困惑度，但通常落在低熵区域，表明多样性降低；而 512 维瓶颈保持更高的熵，但生成困惑度明显更差。128 维瓶颈提供了最佳整体平衡，在保持合理熵的同时取得很强的生成困惑度。因此我们把瓶颈维度设为 128 作为默认设定。这一发现也与先前工作 [37] 一致，它观察到适当的瓶颈能提升性能。

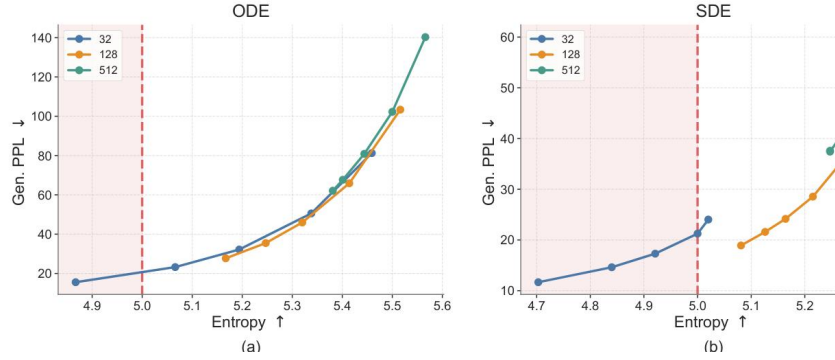


图 12：瓶颈维度的效果。我们在 ODE 与 SDE 采样下对比 32、128 与 512 的瓶颈维度。128 这一适中的瓶颈维度提供最佳的生成困惑度-熵权衡，而过小或过大的瓶颈要么降低多样性，要么损害生成困惑度。红色表示生成质量较差的区域，即熵低于 5。

D.3 去噪模式概率

由于 ELF 通过共享权重的去噪器-解码器同时用 MSE 与 CE 损失训练，每个训练步要么分配给去噪模式，要么分配给解码模式。去噪模式概率控制这种分配：更高的概率强调学习连续去噪动力学，而更低的概率为把嵌入映射回词元提供更多监督。我们通过在训练中改变去噪模式概率来研究这种权衡。

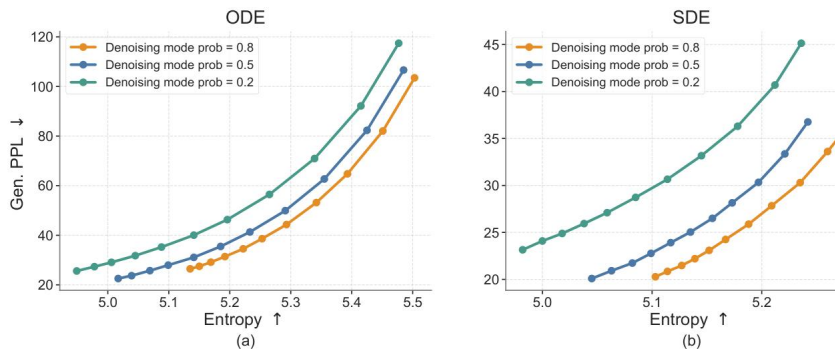


图 13：训练中去噪模式概率的效果。该概率控制共享权重去噪器-解码器模型中去噪与解码更新的分配。去噪模式概率为 0.8 时在 ODE 与 SDE 两种采样器下都提供最佳的生成困惑度-熵权衡。

D.4 条件策略

如 3.3 节所述，我们的模型以时间步、CFG 尺度与模型模式为条件。我们对这些信号采用上下文内条件，即把它们作为条件词元前置到输入序列，使模型能通过完整注意力关注它们。这与传统的 adaLN-Zero 条件设计不同，后者通常会引入额外的模型组件来处理条件输入。我们在图 14 中对比这两种设计。上下文内条件表现略好，同时避免了 adaLN-Zero 引入的显著参数开销（ELF-B 的参数数从 148M 降到 105M）。因此我们把上下文内条件作为默认设定。

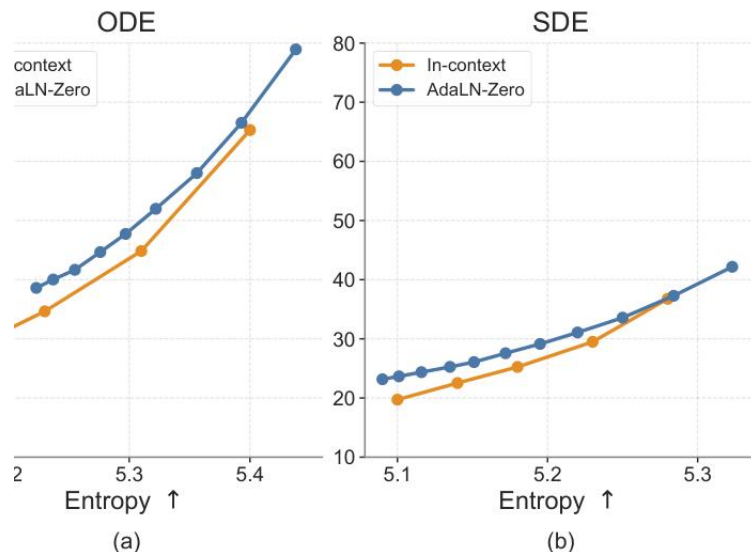


图 14: 条件策略的效果。我们对比上下文内条件与 adaLN-Zero 条件。上下文内条件略微提升性能，同时大幅减少模型参数数量。

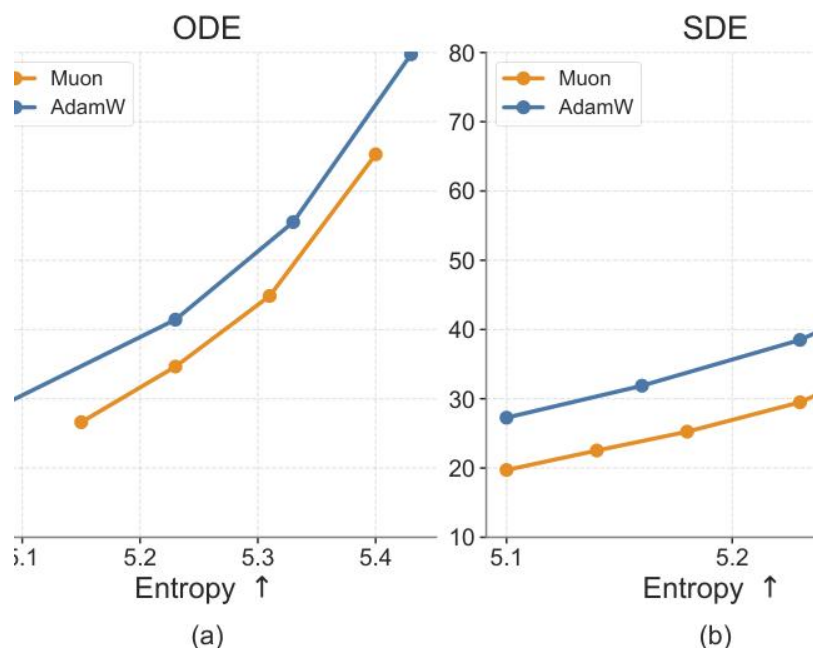


图 15: 优化器的效果。我们使用 Muon 与 AdamW 对比不同优化器下的生成质量。在 ODE 与 SDE 两种采样方法下，Muon 在可比熵下取得更低的生成困惑度。

D.5 优化器

我们评估优化器选择的影响，比较 Muon [33] 与 AdamW [44]，结果显示在图 15 中。我们为两种优化器都调参以获得其最佳性能：对 Muon，我们使用 2×10^{-3} 的学习率；对 AdamW，我们使用 1×10^{-4} 的学习率， $\beta_1 = 0.9$ ， $\beta_2 = 0.95$ 。在相同的步数内，Muon 在训练时达到更低的损失。在推理时，用 Muon 训练的模型在两种采样器下都一致地取得比用 AdamW 训练的模型更好的生成困惑度-熵权衡。这一改进在 SDE 采样下尤其显著，此时 Muon 在相同熵水平达到更低的生成困惑度。这些结果凸显了

优化器选择的重要性。不过，用两种优化器训练的模型仍然都优于其他基线，表明 ELF 的强劲性能不能仅归因于优化器。

D.6 采样方法

我们研究两个能提升推理效率与生成质量的采样设计选择：采样时间调度与随机的、受 SDE 启发的采样。logit-normal 时间调度通过减少所需的去噪步数来提升采样效率，而 SDE 噪声再注入尺度则提供对生成困惑度-熵权衡的额外控制。

时间调度。

默认情况下，我们在推理时使用 logit-normal 时间调度 [34]。我们还评测了一种替代的均匀调度。图 16a 显示了在 ODE 采样下、不同采样步数时时间调度的影响。在所有步数下，logit-normal 调度相比均匀调度一致地降低生成困惑度。这一改进在少步数区域尤其显著。这些结果表明，logit-normal 时间调度提升了采样效率与最终样本质量，很可能是因为使推理时的轨迹与训练时的调度更好地对齐，并把更多采样步分配给噪声更强的时间步。

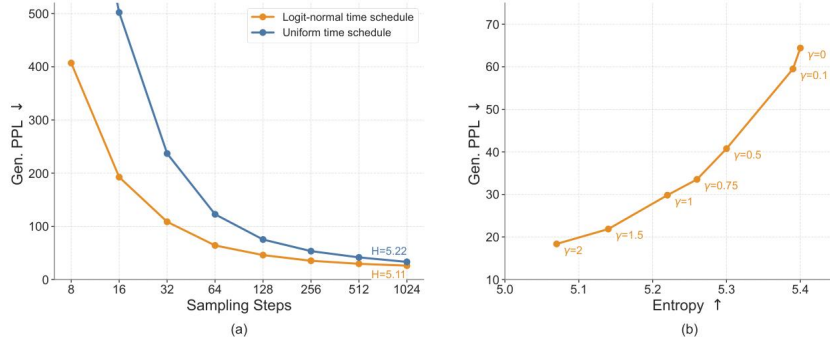


图 16：时间调度与 SDE 噪声再注入尺度的效果。(a) logit-normal 时间调度在不同采样预算下一致地改善生成困惑度，尤其在少步数区域。(b) SDE 噪声再注入尺度 γ 通过调整采样时注入的随机噪声量来控制生成困惑度-熵权衡。

SDE 噪声再注入尺度。

对于 SDE 采样，我们引入一个噪声再注入尺度超参数 γ ，来控制每个采样步注入的随机性大小，如 C.2 节所述。直观上，增大 γ 会引入更多随机性，而 $\gamma = 0$ 则退化为确定性的 ODE 采样。如图 16b 所示， γ 控制生成困惑度-熵权衡：在适中范围内，更大的 γ 导致更低的生成困惑度，同时略微降低熵。

我们假设噪声再注入过程有助于校正早期去噪误差，而不是像 ODE 采样那样确定性地放大不完美的轨迹。因此我们选择 $\gamma = 1.0$ 作为默认设定，它在生成困惑度与熵之间提供了很强的平衡。

D.7 条件生成上的 CFG

我们进一步研究 CFG 尺度对条件生成任务的效果。如图 17 所示，把 CFG 尺度从 1 增到 2 会显著提升 WMT14 De-En 与 XSum 两个任务上的表现，表明更强的条件有助于模型更好地遵循源输入。然而，进一步增大尺度会导致性能逐渐下降，表明过强的引导可能损害生成质量。基于这一趋势，我们把 CFG 尺度 2 作为条件生成的默认设定。

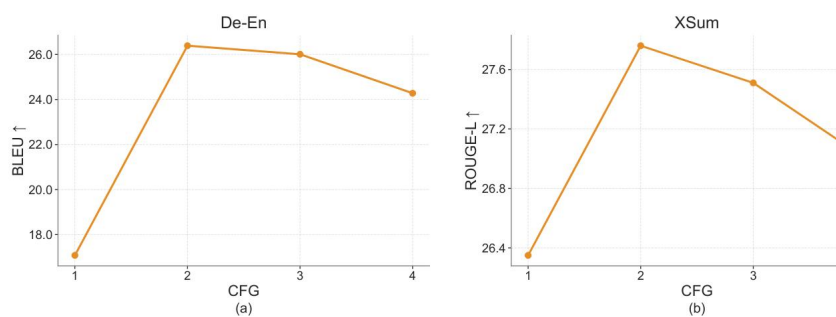


图 17: CFG 尺度对条件生成的效果。我们在 WMT14 De-En 翻译与 XSum 摘要上扫描 CFG 尺度。适度的引导能显著提升任务性能, CFG 尺度 2 在两个任务上都取得最佳效果, 而过强的引导会轻微损害性能。

附录 E 实验细节

E.1 模型架构

我们的模型使用标准的扩散 Transformer 架构 [56]。我们还融入了流行的通用改进，包括 SwiGLU [69]、RMSNorm [89]、RoPE [76] 与 qk-norm [28]。我们使用上下文内条件而非 adaLN-Zero [56] 条件，这使我们能大幅减少参数量；例如，ELF-B 模型大小从 148M 降到 105M 参数。表 6 汇总了 ELF 在不同模型规模下的配置。我们报告了 Transformer 深度、隐藏大小、注意力头数与参数数量，还报告了每个变体在 OWT 数据集上训练使用的 epoch 数。在我们的设定下，较大的模型往往学习更快，因此需要的训练 epoch 更少。

Model	Depth	Hidden size	# Heads	Params	Training epochs
ELF-B	12	768	12	105M	5
ELF-M	24	1056	16	342M	4
ELF-L	32	1280	16	652M	3

表 6: 不同规模的 ELF 模型配置。

E.2 超参数

ELF 流程超参数。

表 7 汇总了 ELF 流程中使用的主要超参数，涵盖模型架构、扩散设定、条件与引导，以及优化细节。除非另有说明，论文中的所有实验都遵循此默认配置。我们纳入这些设定以保持完整性并便于复现。

Model Architecture		Denoising and Decoding Config	
Model	ELF-B	Time schedule	logit normal
Model size	105M	Denoiser ($P_{\text{mean}}, P_{\text{std}}$)	(−1.5, 0.8)
Encoder backbone	T5-small	Denoiser noise scale	2.0
Embedding dimension	512	Decoder ($P_{\text{mean}}, P_{\text{std}}$)	(0.8, 0.8)
Bottleneck dimension	128	Decoder noise scale	5.0
Model dimension	768	Denoiser vs. decoder prob.	0.8 vs. 0.2
Sequence length	1024		
Conditioning and Guidance		Optimization and Training	
Self-conditioning probability	0.5	Optimizer	Muon
Self-conditioning CFG range	[0.5, 5]	Learning rate	0.002
Num. of time tokens	4	Weight decay	0
Num. of model-mode tokens	4	Training epochs	5
Num. of CFG tokens	4	Global batch size	512
SDE γ	1.0	Learning rate schedule	constant
		Warmup epochs	0.5
		EMA decay	0.9999
		Training device	TPU v5p × 64
		Training time	1.5 h per epoch

表 7: ELF-B 在 OpenWebText 数据集上的默认训练超参数与设定。除非另有说明，论文中的所有实验都遵循此默认配置。

系统级对比的推理时设定。

对于图 7 中的系统级对比，我们对所有步数预算都使用带时间调度的 SDE 采样。对 8、16 与 32 步生成，我们把 CFG 尺度设为 3。对于 SDE 采样，我们在 8 与 16 步这种极少数步区域使用更强的噪声注入尺度 $\gamma = 2$ ，并在 32 步时降到 $\gamma = 1.5$ ，因为更长的去噪轨迹需要更少的随机校正。对于表 1 中的系统级对比，我们使用带时间调度的 64 步 ODE 采样。我们把自条件 CFG 尺度设为 1，输入条件 CFG 尺度设为 2。

系统级对比的训练词元预算。

表 8 报告了 ELF 与图 7c 中各基线所使用的估算有效训练词元。我们把基础训练词元估算为 batch 大小 $\times$ 步数 $\times$ 序列长度，并在适用时再加上蒸馏或流映射阶段。OWT 数据集约含 9.04B 词元。按照我们 5 个 epoch 的默认训练调度，ELF 因此使用 45.2B 有效训练词元。所以 ELF 所需的有效训练词元大约比所对比的 DLM 少一个数量级。

Method	Base training	Distillation training	Effective tokens	Ratio
MDLM [63]	$512 \times 1\text{M} \times 1024$	-	524.3B	$11.6\times$
Duo [64]	$512 \times 1\text{M} \times 1024$	-	524.3B	$11.6\times$
MDLM + SDTT [63]	$512 \times 1\text{M} \times 1024$	$512 \times 10\text{K} \times 5 \times 1024$	550.5B	$12.2\times$
Duo + DCD [64]	$512 \times 1\text{M} \times 1024$	$512 \times 10\text{K} \times 5 \times 1024$	550.5B	$12.2\times$
FLM [35]	$512 \times 1\text{M} \times 1024$	-	524.3B	$11.6\times$
FMLM [35]	$512 \times 1\text{M} \times 1024$	$512 \times 100\text{K} \times 1024$	576.7B	$12.8\times$
LangFlow [11]	$512 \times 1\text{M} \times 1024$	-	524.3B	$11.6\times$
ELF (ours)	$5 \times 9.04\text{B}$	-	45.2B	$1.0\times$

表 8: ELF 与先前 DLM 基线（用于图 7c 系统级对比）的估算有效训练词元。我们把基础训练词元估算为 batch 大小 $\times$ 步数 $\times$ 序列长度；在适用处再加上蒸馏/流映射阶段。

E.3 消融研究设定

我们评测 ELF 的几种嵌入表示选择，实现细节如下。我们还尝试了带单独解码器的两阶段训练。除非另有说明，我们保持其他设定与默认 ELF 配置一致。

从零训练的编码器。

我们按照原始 T5-small 训练流程 [60]，在 OpenWebText [20] 上从零训练一个编码器。该编码器训练 5 个 epoch，学习率为 1×10^{-3} ，余弦学习率调度，0.4 epoch warmup，batch 大小为 512。在 ELF 训练期间，我们对编码器输出施加通道级归一化。

E.4 报告数值

系统级对比。

在 6 个独立的评测随机种子下，ELF 表现出高度一致的系统级行为，如表 9 所示。随着采样步数从 8 增到 32，标准误 (SE) 下降。很小的标准误——尤其在 32 步时——表明这些增益对随机种子变化是稳健的，且整体趋势在各次运行中可靠。详细数值见表 9。

Steps	SC CFG	γ	Gen. PPL ↓	Entropy ↑
8	3	2.0	67.32 ± 2.25	5.14 ± 0.085
16	3	2.0	33.66 ± 1.09	5.16 ± 0.026
32	3	1.5	24.08 ± 0.16	5.15 ± 0.002

表 9: 系统级 ELF 性能, 以 6 次独立评测运行 (种子 0-5; $n=6$) 的均值 ± 标准误 (SE) 报告。

带 CFG 尺度的规模化行为。

两种采样方法的默认设定都使用带时间调度的 64 个采样步。对于 SDE 采样器, 我们设 $\gamma = 1.0$ 。精确数值在表 10 中报告。在某一范围内, 更大的 CFG 尺度通过降低 Gen. PPL 来提升生成质量。CFG 尺度的效果在超过 3 之后反转。只有 ELF-L 从把 CFG 尺度从 3 增到 4 中受益。因此在大多数默认消融研究中, 我们只考虑 0.5 到 3 的 CFG 尺度。

Sampler	SC CFG	ELF-B 105M		ELF-M 342M		ELF-L 652M	
		Gen. PPL	Entropy	Gen. PPL	Entropy	Gen. PPL	Entropy
SDE	0.5	36.77	5.28	39.21	5.35	37.50	5.41
	1.0	29.50	5.23	33.45	5.30	31.82	5.37
	1.5	25.25	5.18	28.42	5.26	28.72	5.35
	2.0	22.53	5.14	25.34	5.23	26.47	5.32
	3.0	19.72	5.10	21.69	5.18	23.31	5.28
	3.5	37.56	5.30	36.48	5.34	22.28	5.27
	4.0	36.50	5.29	34.93	5.33	21.37	5.26
ODE	0.5	104.29	5.51	88.51	5.51	68.27	5.52
	1.0	65.30	5.40	62.47	5.44	49.72	5.45
	1.5	44.85	5.31	46.71	5.37	39.97	5.40
	2.0	34.65	5.23	37.66	5.32	33.72	5.36
	3.0	26.62	5.15	28.80	5.24	26.57	5.29

表 10: 不同规模 ELF 模型在 SDE 与 ODE 采样器、64 采样步下的生成困惑度 (Gen. PPL) 与 unigram 熵的规模化性能。自条件 (SC) CFG 尺度的效果在超过 3 之后减弱。

E.5 条件生成

具体而言, AR、MDLM 与 E2D2 的 WMT14 结果取自 E2D2 [4] 论文, SeqDiffuSeq 结果取自 LD4LG [46] 论文, CDCD 结果取自原始 CDCD [15] 论文。对于复现的结果, Duo [64] 使用 Duo 代码库实现⁴, 而 AR、MDLM 与 E2D2 使用 E2D2 代码库复现⁵。

为公平比较, 我们尽可能使用接近其原始实现的设定复现所有基线, 如表 11 所汇总。对于 AR、MDLM 与 E2D2, 我们使用 E2D2 代码库, 并遵循 E2D2 论文在 XSum 上报告的训练与评测配置。注意, 尽管 E2D2 主要为半自回归生成设计, 我们发现 MDLM 也在半自回归设定下 (即块大小 32, 两块生成) 取得最佳性能; 使用不带半自回归生成的单块扩散会降低性能。对于 Duo, 我们以官方 Duo 仓库为起点, 通过添加交叉注意力条件与无分类器引导, 并使用 T5-small 编码器作为条件输入, 来适配到我们的条件生成设定。推理时, 我们生成时不使用半自回归解码。我们调优主要的采样与引导超参数, 并报告得到的、最佳复现结果。

⁴<https://github.com/s-sahoo/duo> ⁵<https://github.com/kuleshov-group/e2d2>

Config	AR	MDLM	E2D2	Duo	
Architecture					
Codebase	E2D2	E2D2	E2D2	Duo	Duo
Tokenizer	Qwen3-0.6B	Qwen3-0.6B	Qwen3-0.6B	T5-small	T5-small
Hidden Size	256	256	256	768	768
Intermediate Size	768	768	768	–	–
#Layers / Blocks	28	28	enc=20, dec=8	12	12
Sequence Length	64	64	64	64	64
Max Cond Length	1024	1024	1024	1024	64
Cond Embed	–	–	–	T5-small	T5-small
Training					
Dataset	XSum	XSum	XSum	XSum	De-En
Learning Rate	3e-4	3e-4	3e-4	3e-4	3e-4
LR Scheduler	const	const	const	const	const
Warmup Steps	1000	1000	1000	2500	2500
Global Batch Size	128	128	128	512	512
Optimizer	DecoupledAdamW	DecoupledAdamW	DecoupledAdamW	AdamW	AdamW
Loss Type	NLL	MDLM ELBO	E2D2 ELBO	Duo ELBO	Duo ELBO
Train Steps	500K	500K	500K	1M	1M
Evaluation					
Sampling Strategy	greedy	predict_and_noise	predict_and_noise	Duo sampler	Duo sampler
Sampling Steps	$L = 64$ (AR)	$\approx L$ (first-hit)	$\approx L$ (first-hit)	1000	1000
Block size	1	32	8	-	-
CFG Scale	–	–	–	1.0	1.5
Checkpoint	best	best	best	best	best
EMA	true	true	true	true	true

表 11: 我们复现的 AR、MDLM、E2D2 与 Duo 基线在条件生成任务上的详细训练与评测配置。AR、MDLM 与 E2D2 在 XSum 上使用 E2D2 [4] 代码库复现，并遵循 E2D2 论文报告的配置。对于 Duo，我们以原始 Duo [64] 仓库为基础，添加交叉注意力条件与 CFG，适配 T5-small 编码器以匹配我们的设定，并调优超参数以获得最强的复现结果。

附录 F 定性示例

F.1 去噪轨迹

图 18 可视化 ELF 去噪过程中的中间预测。从 $t = 0$ 的重复词元开始，随着 t 接近 1，模型逐渐形成语义上有意义的短语，改善语法，并细化用词。这一轨迹说明了连续扩散生成如何逐步把解码为乱码文本的带噪嵌入，转化为解码为合语法句子的干净嵌入。

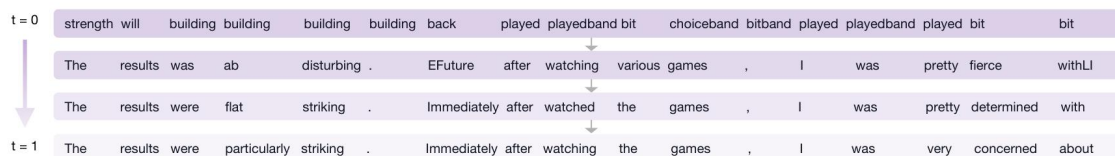


图 18: ELF-B 的去噪轨迹。随着 t 从 0 增到 1，不合语法的句子逐步被细化为流畅、合语法的文本。

F.2 OpenWebText 上的无条件生成示例

我们给出 ELF-B 在 OpenWebText 上生成的三个无条件样本，并报告它们的熵与生成困惑度（Gen. PPL）。这些示例表明 ELF 能跨不同领域生成流畅、句法连贯且主题一致的长文本。

【样本 1 · 熵: 5.36 Gen. PPL: 21.04】该论文文本较长，此处保留原文（英文）。

【样本 2 · 熵: 5.27 Gen. PPL: 21.29】该论文文本较长，此处保留原文（英文）。

【样本 3 · 熵: 5.17 Gen. PPL: 21.80】该论文文本较长，此处保留原文（英文）。

F.3 条件生成示例

WMT14 De-En 定性示例。

我们给出 WMT14 De-En 上的定性示例，以补充语料级 BLEU 结果。ELF 通常能生成流畅且全局连贯的翻译。

（原文、参考译文与 ELF 生成的示例文本见附录 F.3，此处保留英文原文。）

XSum 定性示例。

我们给出 XSum 上的定性示例，以补充 ROUGE 结果。ELF 通常能生成流畅且简洁的摘要，准确捕捉源文档的主要内容。

F.4 ELF+PD 在 OpenWebText 上的无条件生成示例

我们给出最终轮单步蒸馏的 ELF+PD 模型在每个采样步下的一个样本，并报告其熵与生成困惑度（Gen. PPL）。